ԵՐԵՎԱՆԻ ՊԵՏԱԿԱՆ ՀԱՄԱԼՍԱՐԱՆ

ՄԱԹԵՄԱՏԻԿԱՅԻ ԵՎ ՄԵԽԱՆԻԿԱՅԻ ՖԱԿՈՒԼՏԵՏ

ԱԿՏՈՒԱՐԱԿԱՆ ԵՎ ՖԻՆԱՆՍԱԿԱՆ ՄԱԹԵՄԱՏԻԿԱՅԻ ԱՄԲԻՈՆ

ԲԼՈԿՉԵՅՆ ԵՎ ԹՎԱՅԻՆ ԱՐԺՈՒՅԹՆԵՐ ՄԱԳԻՍՏՐՈՍԱԿԱՆ ԿՐԹԱԿԱՆ ԾՐԱԳԻՐ

ԱՍԱՏՈՒՐՅԱՆ ՆԵՐՍԵՍ ԱՐՏԱՎԱԶԴԻ

ՄԱԳԻՍՏՐՈՍԱԿԱՆ ԹԵԶ

ԷԼԵԿՏՐՈՆԱՅԻՆ ՔՎԵԱՐԿՈՒԹՅԱՆ ՀԱՄԱԿԱՐԳԵՐ

«Բլոկչեյն և թվային արժույթներ» մասնագիտացմամբ ֆինանսական մաթեմատիկայի մագիստրոսի որակավորման աստիճանի հայցման համար

ԵՐԵՎԱՆ 2026

Ուսանող`

Ասատուրյան Ներսես Արտավազդի

Գիտական ղեկավար`

ֆ.մ.գ.թ. Դավիդովա Դիանա

«Թույլատրել պաշտպանության»

Ամբիոնի վարիչ`

տնտեսագիտության թեկնածու, դոցենտ Ավետիսյան Կարինե

«_____»_____20_____թ.

Էլեկտրոնային քվեարկության համակարգեր

Electronic Voting Systems

Системы электронного голосования

ABSTRACT

Goal. The goal of this work is to study modern electronic voting systems, analyze in detail the Aura private voting protocol (Jivanyan, Feickert, 2022), and deliver its first open-source implementation with practical performance benchmarks. The work aims to demonstrate that the Aura protocol is practically applicable to digital direct-democracy systems.

Results. The first open-source implementation of the Aura protocol has been developed and is publicly available on GitHub. Every phase of the implementation — distributed key generation, voter registration, encryption, zero-knowledge proofs, and threshold decryption — has been benchmarked under varying parameters. A step-by-step numerical example of the Aura protocol is also included to make the abstract construction concrete.

Code is available at:
https://github.com/nerses-asaturyan/aura

Contents

INTRODUCTION

Motivation

Democratic elections are one of the most fundamental institutions of modern society, yet the process of casting and counting votes still relies heavily on paper ballots, trusted intermediaries, and centralized election authorities. As digital infrastructure becomes ubiquitous, there is growing interest in moving elections — and more generally, collective decision-making processes — onto cryptographic protocols that can be verified by anyone, while still protecting the privacy of individual voters.

Electronic voting (e-voting) systems aim to provide three guarantees that are notoriously hard to achieve simultaneously: **ballot privacy** (no one learns how a specific voter voted), **verifiability** (anyone can check that the tally is correct), and **eligibility** (only registered voters can vote, and each voter can vote at most once). Modern cryptographic e-voting schemes attempt to satisfy all three using building blocks like homomorphic encryption, zero-knowledge proofs, threshold cryptography, and blockchain-based bulletin boards.

Beyond traditional government elections, the same machinery enables a much broader vision: **direct digital democracy**, in which citizens vote frequently on individual policy questions rather than electing representatives who decide on their behalf. For such systems to be trusted at scale, they must be both cryptographically sound and practically efficient.

Goal and Objectives

The goal of this thesis is to study modern cryptographic e-voting systems, with a focus on the *Aura private voting protocol* of Jivanyan and Feickert (2022) [1], and to deliver the first open-source implementation of the protocol along with practical performance benchmarks.

The specific objectives are:

- Survey existing cryptographic e-voting systems briefly to position Aura among them.
- Provide a self-contained presentation of the cryptographic primitives used in Aura (ElGamal encryption, Pedersen commitments, Merkle trees, Schnorr proofs, one-of-many proofs, threshold secret sharing).
- Describe the full Aura protocol from setup through tally, with a step-by-step worked numerical example.
- Implement the protocol end-to-end in code and publish it as an open-source repository.
- Benchmark each component of the protocol under varying parameters (number of voters, candidates, threshold size) to evaluate practical viability.

Object and Subject of Study

The **object** of study is cryptographic electronic voting systems. The **subject** is the Aura protocol — its construction, security guarantees, and concrete computational cost when implemented.

Hypothesis

The Aura protocol, despite combining several heavyweight cryptographic primitives (threshold ElGamal, one-of-many proofs, range proofs), is practically efficient enough to support real-world voting scenarios — including frequent direct-democracy referenda — on commodity hardware.

Theoretical and Methodological Foundations

The work draws on results from public-key cryptography, zero-knowledge proof systems, and distributed computing. Methodologically, the thesis combines (i) a brief review of existing e-voting schemes, (ii) a formal exposition of the Aura protocol, (iii) a software-engineering effort to implement the protocol, and (iv) empirical performance measurement.

Scientific and Practical Significance

Scientifically, this thesis contributes the first open and reproducible implementation of the Aura protocol, enabling other researchers to verify the authors' claims, build on the codebase, or compare against new schemes. It also includes a worked numerical example of Aura that makes the protocol's mechanics easier to follow.

Practically, the benchmarks provide concrete guidance to anyone considering deploying Aura — for example, in DAO governance, university student elections, or municipal direct-democracy pilots — by quantifying the cost of each protocol step.

Thesis Structure

The remainder of this thesis is organized as follows. Chapter 1 briefly reviews related work in cryptographic e-voting. Chapter 2 introduces the cryptographic primitives required to understand Aura. Chapter 3 describes the Aura protocol in full. Chapter 4 walks through the protocol with a numerical example. Chapter 5 documents the open-source implementation. Chapter 6 reports benchmark results. Chapter 7 discusses limitations and future work, and the final chapter concludes.

1. RELATED WORK

Cryptographic e-voting has been studied for several decades, and several practical systems have been deployed at varying scales. Helios [2] is among the earliest deployed cryptographic e-voting systems and uses ElGamal encryption with cast-or-audit ballot verification; it has been used for university and IACR elections, although it relies on a small set of trusted election servers and does not provide coercion resistance. The Russian federal remote e-voting scheme deployed in 2021 [3] is a more recent large-scale deployment built on a blockchain backend and blind signatures, but has been criticized for limited transparency and verifiability. Broader surveys of the field include [4] on the cryptographic side and [5] on the blockchain-based side, both of which categorize existing systems into mix-net-based, homomorphic-tally-based, and blind-signature-based families.

Aura [1] was introduced in 2022 by Jivanyan and Feickert as a private voting protocol with reduced trust assumptions on the tallying authorities. Compared to Helios, it adds threshold decryption and Groth-Kohlweiss one-of-many proofs for stronger anonymity; compared to most blockchain-based schemes, it does not require a smart-contract platform — only a public bulletin board. To date, no public implementation of Aura exists, which motivates the engineering contribution of this thesis.

2. CRYPTOGRAPHIC PRIMITIVES

Aura is built from several standard cryptographic primitives. This chapter gives a self-contained introduction to each, sufficient to follow the protocol description in Chapter 3. Readers familiar with public-key cryptography may skim this chapter.

2.1 Cyclic Groups and the Discrete Logarithm Problem

The cryptographic primitives used in Aura are all built on top of a finite cyclic group in which the discrete logarithm problem is believed to be intractable. Throughout this thesis, $\mathbb{G}$ denotes a cyclic group of prime order q with generator g, written multiplicatively, so that

$$\mathbb{G} = \{g^0, g^1, g^2, \ldots, g^{q-1}\},$$

and exponents live in the finite field $\mathbb{Z}_q$. Working in a prime-order group means every non-identity element is itself a generator and admits cleaner security proofs.

Given g and $h = g^x$, the *discrete logarithm problem* (DLP) is to recover $x \in \mathbb{Z}_q$. Computing h from x takes only $O(\log q)$ group multiplications, but the reverse direction is believed to be hard when q is large; the best known generic algorithms run in time roughly $\sqrt{q}$, which is infeasible once q has on the order of 256 bits. This is what makes g^x usable as a one-way commitment to x — a property Aura relies on, most directly in the serial number $\text{sn}_i = u^{s_i}$.

A stronger but related hardness assumption, used by ElGamal encryption in Aura, is the *Decisional Diffie-Hellman* (DDH) assumption: given g, g^a, g^b for random $a, b \in \mathbb{Z}_q$, no efficient algorithm can distinguish g^{ab} from a uniformly random element of $\mathbb{G}$.

In practice, $\mathbb{G}$ is realized either as a prime-order subgroup of $\mathbb{Z}_p^*$ (the setting of the worked example in Chapter 4, with toy parameters $p = 47$, $q = 23$) or as an elliptic-curve group (the setting of the implementation in Chapter 5). Elliptic-curve groups achieve the same security level at much smaller key sizes, which is why every modern deployment of discrete-log-based cryptography uses them.

2.2 ElGamal Encryption

ElGamal encryption [10] is a public-key cryptosystem whose security reduces to the DDH assumption in $\mathbb{G}$. Aura uses it as the encryption layer for every ballot.

Standard ElGamal

The secret key is a uniformly random $sk \in \mathbb{Z}_q$, and the public key is $pk = g^{sk} \in \mathbb{G}$. To encrypt a message $m \in \mathbb{G}$ with fresh randomness $r \in \mathbb{Z}_q$, the sender computes

$$\text{Enc}_{pk}(m; r) = (c_1, c_2) = (g^r,\ m \cdot pk^r).$$

Decryption recovers m as $m = c_2 / c_1^{sk}$, since $c_1^{sk} = g^{r \cdot sk} = pk^r$. ElGamal is *multiplicatively homomorphic*: the componentwise product of two ciphertexts encrypts the product of the corresponding plaintexts,

$$\text{Enc}_{pk}(m_1) \cdot \text{Enc}_{pk}(m_2) = \text{Enc}_{pk}(m_1 \cdot m_2).$$

Exponential ElGamal

For voting, an additive homomorphism is needed, so that ciphertexts of $0/1$ votes can be multiplied to encrypt their sum. *Exponential ElGamal* achieves this by encoding an integer message $v \in \mathbb{Z}_q$ as the group element g^v before encryption:

$$\text{Enc}_{pk}(v; r) = (g^r,\ g^v \cdot pk^r).$$

The componentwise product of two such ciphertexts now encrypts $g^{v_1+v_2}$, turning the multiplicative homomorphism into an additive one. The cost is that decryption recovers only g^v, not v itself — but when v is known to lie in a small range (such as a vote tally $T_j \in [0, n]$), v can be read off by lookup against a precomputed table $\{g^0, g^1, \ldots, g^n\}$.

This is the variant Aura uses: each ballot slot encrypts a bit $v_{i,j} \in \{0, 1\}$ as $E_{i,j} = (g^{\rho_{i,j}}, g^{v_{i,j}} \cdot pk^{\rho_{i,j}})$, and the homomorphic column product $E_\Sigma^{(j)} = \prod_i E_{i,j}$ then encrypts the candidate total $T_j = \sum_i v_{i,j}$.

2.3 Pedersen Commitments

A *commitment scheme* lets a party publish a short value that binds them to a secret without revealing it, and later open the commitment to prove what was committed. Pedersen commitments [8] are the flavor used in Aura, because they are both *perfectly hiding* and *computationally binding* under the discrete logarithm assumption.

Definition

Fix two generators $g, h \in \mathbb{G}$ such that no party knows the discrete logarithm of h with respect to g. To commit to a value $v \in \mathbb{Z}_q$, the committer samples a fresh random $r \in \mathbb{Z}_q$ (the *blinding factor*) and publishes

$$\text{cm}(v, r) = g^v h^r \in \mathbb{G}.$$

To open the commitment later, the committer reveals (v, r), and any verifier checks that $g^v h^r$ matches the published $\text{cm}(v, r)$.

Security Properties

Perfectly hiding. Because r is uniform in $\mathbb{Z}_q$ and multiplication by h^r is a bijection on $\mathbb{G}$, the commitment $\text{cm}(v, r)$ is uniformly distributed in $\mathbb{G}$ regardless of v. No adversary, even with unbounded computing power, learns anything about v from $\text{cm}(v, r)$.

Computationally binding. Suppose a committer could open the same commitment to two distinct pairs $(v, r) \neq (v', r')$. Then $g^v h^r = g^{v'} h^{r'}$, which rearranges to $h = g^{(v-v')/(r'-r)}$, revealing the discrete logarithm of h with respect to g. Under the DLP assumption (Section 2.1), this is computationally infeasible, so the committer is bound to her original choice of v.

These two properties trade off against each other: hiding is unconditional (perfect), while binding holds only against computationally bounded adversaries. This trade-off is fundamental — no commitment scheme can be both perfectly hiding and perfectly binding at once.

Why Aura Uses Pedersen Commitments

In Phase 2 of the protocol, each voter V_i registers by publishing $\text{cm}_i = g^{s_i} h^{r_i}$, where s_i is her private seed and r_i is fresh randomness. Both properties of Pedersen commitments are essential:

- *Perfect hiding* guarantees that the public registration list $L = (\text{cm}_1, \ldots, \text{cm}_n)$ leaks no information about any voter's seed — even an attacker with unbounded resources cannot link a commitment to a specific s_i.
- *Computational binding* guarantees that a voter cannot later claim a different seed than the one she registered with, which would let her vote multiple times under different identities or forge eligibility proofs.

The commitment is also *additively homomorphic*: $\text{cm}(v_1, r_1) \cdot \text{cm}(v_2, r_2) = \text{cm}(v_1 + v_2, r_1 + r_2)$. Aura does not use this property in the registration phase, but it is exploited inside the Groth-Kohlweiss one-of-many proof (Section 2.7) that proves a voter's commitment lies in L without revealing which one.

2.4 Merkle Trees

A *Merkle tree* [6] is a hash tree that compresses a list of values into a single short hash — the *root* — in such a way that membership of any individual value can be proven with only $O(\log n)$ data. Aura itself does not use a Merkle tree, but the construction is included here as standard background, and because the worked numerical example in Chapter 4 introduces one as a pedagogical simplification.

Construction

Let H_{MT} be a collision-resistant hash function. Given a list of n leaves $(\ell_1, \ldots, \ell_n)$, the tree is built bottom-up: each internal node is the hash of its children,

$$\text{parent} = H_{\text{MT}}(\text{children}),$$

and the topmost node is the root, denoted MT.root. The root is a short ($O(1)$ size) commitment to the full list: changing any leaf, anywhere in the tree, changes the root with overwhelming probability under collision resistance.

Authentication Paths

To prove that a particular leaf ℓ_i is in the tree without revealing the other leaves, the prover supplies the *authentication path*: the sibling hashes encountered on the way from ℓ_i up to the root. A verifier who knows MT.root recomputes the path by hashing ℓ_i together with each set of siblings in turn, and checks that the computed root matches the published one. The path has length $O(\log n)$, so membership proofs take $O(\log n)$ space and time.

Why Aura Does Not Use One

A natural intuition is that Merkle trees are necessary to make voter-list membership proofs scale: with n registered voters, a flat list takes $O(n)$ space, while a Merkle root takes $O(1)$. However, the Groth-Kohlweiss commitment-set proof (Section 2.7) already produces $O(\log n)$-size membership proofs operating directly on a flat list of commitments — without any tree structure on top of it. Aura therefore publishes the full voter commitment list $L = (\text{cm}_1, \ldots, \text{cm}_n)$ and runs the commitment-set proof on L directly, achieving logarithmic proof size without the extra Merkle layer. The flat-list approach also avoids the bookkeeping of authentication paths that voters would otherwise need to keep synchronized with a published root.

2.5 Schnorr Identification and Sigma Protocols

A *sigma protocol* [13] is a three-move interactive proof in which a prover convinces a verifier that she knows a witness w for a public statement y — without revealing anything about w — by exchanging a *commitment*, a *challenge*, and a *response*. Schnorr's protocol [11] is the canonical example: it proves knowledge of a discrete logarithm.

Schnorr's Protocol

The prover P knows $x \in \mathbb{Z}_q$, the verifier V knows the statement $y = g^x \in \mathbb{G}$, and they wish to convince V that P knows x. The interaction is:

1. P samples $r \leftarrow \mathbb{Z}_q$, sends the commitment $t = g^r$.

2. V samples a random challenge $c \leftarrow \mathbb{Z}_q$ and sends it.

3. P replies with the response $z = r + cx \mod q$.

The verifier accepts iff $g^z = t \cdot y^c$, which holds for an honest prover since $g^z = g^{r+cx} = t \cdot y^c$.

Security Properties

Schnorr's protocol satisfies two properties that together define a sigma protocol. *Special soundness:* from any two accepting transcripts (t, c, z) and (t, c', z') with the same commitment but different challenges, the witness can be extracted as $x = (z - z')/(c - c') \mod q$. A cheating prover that succeeds non-negligibly could be rewound to recover x, contradicting the DLP. *Honest-verifier zero knowledge:* a simulator can produce transcripts indistinguishable from real ones by picking c, z uniformly and setting $t = g^z \cdot y^{-c}$, so the protocol leaks nothing about x.

The Fiat-Shamir Transform

The protocol above is interactive. The *Fiat-Shamir transform* [12] makes it non-interactive by replacing the verifier's challenge with a hash of the statement and the commitment,

$$c = H(y,\ t),$$

where H is modeled as a random oracle (Section 2.9). The prover produces the entire transcript on her own and sends (t, z); any verifier with y recomputes $c = H(y, t)$ and checks $g^z = t \cdot y^c$. Soundness in the random oracle model rests on a specific property: because the prover must commit to t before seeing c, and because H behaves as a truly random function, she cannot manipulate t to make a particular c come out of the hash — so the hash output is, from her perspective, indistinguishable from a freshly-drawn random challenge. This is the construction used throughout Aura: every interactive proof in the protocol is a sigma protocol made non-interactive in exactly this way.

2.6 Disjunctive Proofs (OR Proofs)

Sigma protocols prove a single statement at a time. For voting, what is needed is a proof that *one of two* statements is true, without revealing which one. The *Cramer-Damgård-Schoenmakers transformation* [13] composes any two sigma protocols into such a disjunctive (OR) proof.

Construction

Suppose the prover wants to prove $\text{Stmt}_0 \vee \text{Stmt}_1$ knowing a witness only for the true branch $b \in \{0, 1\}$. The trick is to use the simulator (from honest-verifier zero knowledge in Section 2.5) to fake the false branch, while running the real protocol on the true one:

1. For the false branch $1-b$, pick a random c_{1-b} and run the simulator to produce $(t_{1-b}, c_{1-b}, z_{1-b})$.

2. For the true branch b, compute the real commitment t_b.

3. Send (t_0, t_1); the verifier replies with a single challenge c.

4. Set $c_b = c - c_{1-b}$ and compute the real response z_b.

5. Send (c_0, c_1, z_0, z_1).

The verifier checks that $c_0 + c_1 = c$ and that both sub-transcripts verify. The split of c into (c_0, c_1) binds the two branches: the prover can pre-pick one sub-challenge, but the other is forced by c, so she can only complete the protocol on a branch for which she has a witness. Both branches verify regardless of b, so the verifier learns nothing about which one was real. After Fiat-Shamir, with $c = H(y_0, y_1, t_0, t_1)$, the whole proof is a single message.

Use in Aura: Bit Proofs

Each ballot ciphertext slot $E_{i,j} = (c_{1,j}, c_{2,j})$ should encrypt either 0 or 1, but a malicious voter could try to encrypt a larger value to give a candidate extra votes. The *bit proof* forces $v_{i,j} \in \{0, 1\}$ via an OR of two Schnorr-style statements:

$$\underbrace{c_{2,j} = pk^{\rho_{i,j}}}_{v_{i,j}=0} \quad \text{OR} \quad \underbrace{c_{2,j}/g = pk^{\rho_{i,j}}}_{v_{i,j}=1}.$$

The voter knows $\rho_{i,j}$ for exactly one branch (the one matching her actual bit) and uses the OR construction above to produce a proof that verifies for both — without revealing which.

2.7 One-of-Many Proofs (Groth-Kohlweiss)

The OR proof of Section 2.6 extends to disjunctions of n statements, but at a cost: proof size grows linearly in n. For an Aura election with thousands of voters, this is infeasible. The *Groth-Kohlweiss one-of-many proof* [7] solves this: it proves that one of n public commitments opens to zero, in proof size $O(\log n)$, without revealing which.

Construction Sketch

The prover holds commitments $(\mathrm{cm}_1, \ldots, \mathrm{cm}_n)$ and an index ℓ such that she knows an opening of cm_ℓ to zero. Instead of running n disjunctive branches, she commits to the bits of ℓ in binary, $\ell = (\ell_1, \ldots, \ell_m)$ with $m = \lceil \log_2 n \rceil$, and runs m parallel Schnorr-style proofs that each bit commitment opens to 0 or 1. The challenge-response phase mixes these bit commitments with the full list in a way that algebraically cancels every term except the ℓ-th, leaving exactly the statement that cm_ℓ opens to zero. The prover transmits only $O(\log n)$ group elements and scalars in total.

Use in Aura

At ballot construction, each voter must prove she is one of the registered voters in $L = (\mathrm{cm}_1, \ldots, \mathrm{cm}_n)$ without revealing her index. Aura applies the Groth-Kohlweiss proof to a shifted list of commitments encoding her seed s_i and serial number $\mathrm{sn}_i = u^{s_i}$, so that the same proof simultaneously establishes eligibility and consistency between cm_i and sn_i. This is the core anonymity primitive of the protocol: it is what keeps ballots small while still hiding the voter's identity.

2.8 Shamir Secret Sharing and Threshold Cryptography

Shamir's secret sharing [9] splits a secret into n shares such that any t shares can reconstruct the secret, but fewer than t shares reveal nothing about it. *Threshold cryptography* extends this idea so that the shareholders can jointly use the secret — for instance, to decrypt a ciphertext — without ever assembling it in one place. Aura uses both techniques to distribute the ElGamal decryption key across η tallying authorities.

Shamir's Scheme

To share a secret $s \in \mathbb{Z}_q$ with threshold t, the dealer samples a random polynomial of degree $t-1$ over $\mathbb{Z}_q$ with constant term s:

$$f(x) = s + a_1 x + a_2 x^2 + \cdots + a_{t-1} x^{t-1}.$$

Share i is $f(i)$, given to party i. Any t shares determine f uniquely by Lagrange interpolation, recovering $s = f(0)$. With fewer than t shares, s is information-theoretically hidden: every value of s is consistent with the missing shares.

Verifiable Secret Sharing

Plain Shamir sharing assumes an honest dealer. *Feldman verifiable secret sharing* [14] adds public commitments to the polynomial coefficients: the dealer publishes $C_k = g^{a_k}$ for $k = 0, \ldots, t-1$ (with $a_0 = s$), allowing each shareholder i to verify that her share satisfies

$$g^{f(i)} \stackrel{?}{=} \prod_{k=0}^{t-1} (C_k)^{i^k}.$$

A cheating dealer cannot distribute inconsistent shares without being caught. The trade-off is that $C_0 = g^s$ is now public, which leaks g^s but not s itself — exactly what is needed when g^s will be used as a public key.

Threshold ElGamal

Aura combines these ideas to generate the ElGamal keypair without trusting any single party. Each authority A_j acts as a dealer for its own random polynomial f_j and distributes shares $f_j(i)$ to every

authority A_i. Each A_i sums the shares she receives,

$$sk_i = \sum_j f_j(i) \mod q,$$

giving her a Shamir share of the joint secret $sk = \sum_j f_j(0)$, which is never computed in one place. The public key is $pk = g^{sk} = \prod_j C_{j,0}$ and is published.

To decrypt a ciphertext (c_1, c_2), any t authorities S each publish a partial decryption $d_i = c_1^{sk_i}$ along with a NIZK proof that this share was computed correctly. The combiner reconstructs c_1^{sk} by Lagrange interpolation in the exponent,

$$c_1^{sk} = \prod_{i \in S} d_i^{\lambda_i}, \quad \lambda_i = \prod_{\substack{j \in S \\ j \neq i}} \frac{-j}{i-j} \mod q,$$

and recovers the plaintext as $m = c_2 / c_1^{sk}$. The full secret key sk never appears anywhere; only its action on c_1 is reconstructed, and only for the specific aggregate ciphertext being decrypted. This is exactly what lets Aura decrypt the homomorphic tally without giving any single authority — or any group smaller than t — the power to decrypt individual ballots.

2.9 The Random Oracle Model

The *random oracle model* (ROM) [15] is a proof framework, not a construction. In it, a cryptographic hash function H is idealized as a uniformly random function: every fresh input produces an independent, uniformly distributed output, and the same input always produces the same output. Security proofs in the ROM treat H as a public black box that any party can query, and reason about adversaries in terms of how they interact with this idealized oracle.

Real hash functions like SHA-256 are deterministic algorithms with internal structure, not random functions. The ROM is therefore a heuristic: a proof in the ROM does not directly imply security in the standard model, and there are contrived constructions that are secure in the ROM but insecure under any concrete instantiation. Despite this, the ROM remains widely used in cryptography because (i) for natural protocols, ROM-based security has empirically held up, and (ii) for many constructions — including Fiat-Shamir-compiled sigma protocols — no comparably efficient construction with a proof in the standard model is known.

In Aura, every non-interactive proof is built using Fiat-Shamir (Section 2.5), and the soundness analysis of each — the bit proofs, the range proof, the Groth-Kohlweiss one-of-many proof, and the proofs of correct decryption — is carried out in the ROM. Concretely, the soundness argument requires that the prover cannot predict $H(y, t)$ before committing to t, which is exactly the property the ROM guarantees: under the model, the hash output is unknown until queried and then fixed for all future queries on the same input.

3. THE AURA VOTING PROTOCOL

This chapter describes the Aura protocol of Jivanyan and Feickert [1] in full. The protocol involves three classes of participants: n voters, η tallying authorities (with threshold t), and a public bulletin board $\mathcal{B}$. The election runs over k candidates, with each voter selecting between ℓ_{min} and ℓ_{max} candidates.

The presentation here uses the canonical notation introduced in Chapter 2: g is the main generator of the group $\mathbb{G}$ of order q, h and u are independent generators (with unknown discrete-log relationship to g), sk and $pk = g^{sk}$ are the threshold ElGamal keypair, $\text{cm}_i = g^{s_i} h^{r_i}$ is voter V_i's Pedersen commitment to her seed s_i with randomness r_i, and $E_{i,j} = (c_{1,j}, c_{2,j})$ is the ElGamal ciphertext of vote slot j on voter i's ballot.

3.1 Protocol Overview

Aura proceeds in ten phases that map onto a small set of actors and deliver, between them, the protocol's three core security properties: *ballot privacy* (no one can read individual votes), *voter anonymity* (no one can link a ballot to a voter), and *universal verifiability* (anyone can audit the result). Table 3.1 summarizes the phases and the security property each contributes.

Table 3.1: Aura protocol phases and the security properties they deliver.

#	Phase	What it delivers
1	Distributed key generation	Threshold pk with no single point of trust.
2	Voter registration	Pedersen commitments hide voter seeds; binding to a public list.
3	Voter commitment list	Public anchor L for eligibility proofs.
4	Encrypted serial number	Per-voter nullifier hidden during voting; unlocked at tally for coercion resistance.
5	Ballot encryption	Each vote slot encrypted under pk; tally is computable homomorphically.
6	Ballot validity proof	NIZK that the ballot is well-formed without revealing it.
7	Ballot publication & verification	Public audit of every ballot before it counts.
8	Homomorphic aggregation	Per-candidate totals as ciphertexts, publicly.
9	Threshold decryption	t authorities reveal the totals, jointly.
10	Tally lookup	Recover plaintext vote counts.

The phase boundaries are pedagogical; the original paper organizes the same logic into a smaller set of named algorithms (SetupElection, SetupTally, SetupVoter, Vote, VerifyBallot, Tally, VerifyTally). The mapping is: Phase 1 corresponds to SetupTally; Phases 2–3 to SetupVoter; Phases 4–6

to Vote; Phase 7 to VerifyBallot; Phases 8–9 to Tally; and Phase 10 to VerifyTally.

3.2 Phase 1: Distributed Key Generation

Aura uses a threshold ElGamal cryptosystem over $\mathbb{G}$ where the η tallying authorities jointly hold the secret key sk in shared form, with no single party knowing it. This phase produces both the public key pk used by voters and each authority's private share sk_i.

The construction is a distributed, verifiable adaptation of Pedersen's threshold scheme, modified to defend against key-cancellation attacks and to ensure publicly-verifiable correctness of every step. Each authority A_α for $1 \leq \alpha \leq \eta$ does the following:

1. Samples coefficients $a_{\alpha,0}, \ldots, a_{\alpha,t-1} \in \mathbb{Z}_q$ uniformly at random and defines a polynomial of degree $t-1$:
$$f_\alpha(x) = \sum_{j=0}^{t-1} a_{\alpha,j} x^j.$$

2. Computes Feldman commitments $C_{\alpha,j} = g^{a_{\alpha,j}}$ for $j = 0, \ldots, t-1$, and broadcasts the vector $\mathbf{C}_\alpha = (C_{\alpha,0}, \ldots, C_{\alpha,t-1})$ together with a NIZK proof of knowledge of the discrete log $a_{\alpha,0}$ (a Schnorr proof, Section 2.5). This proof prevents key cancellation: an authority cannot publish $C_{\alpha,0}$ as a function of other authorities' commitments without knowing the underlying scalar.

3. For each $\beta \neq \alpha$, computes $y_{\alpha,\beta} = f_\alpha(\beta)$ and sends it privately to A_β.

4. On receiving a share $y_{\beta,\alpha}$ from A_β, verifies it against A_β's public commitments by checking
$$g^{y_{\beta,\alpha}} \stackrel{?}{=} \prod_{j=0}^{t-1} (C_{\beta,j})^{\alpha^j}.$$
Aborts if the check fails.

5. Computes its share of the joint secret key as
$$sk_\alpha = \sum_{\beta=1}^{\eta} y_{\beta,\alpha} \mod q,$$
and the corresponding public key share $pk_\alpha = g^{sk_\alpha}$. Posts (α, pk_α) together with a NIZK of correctness to the bulletin board.

The joint public key, computable by anyone from the public commitments, is
$$pk = \prod_{\alpha=1}^{\eta} C_{\alpha,0} = g^{\sum_\alpha a_{\alpha,0}} = g^{sk}.$$

The joint secret $sk = \sum_\alpha a_{\alpha,0}$ exists only implicitly; no party ever computes it. Any t authorities, however, hold a t-of-η Shamir sharing of sk via their sk_α values, so they can later cooperatively decrypt without reconstructing sk in one place.

3.3 Phase 2: Voter Registration

Each registered voter V_i holds a long-term signing keypair (used to authenticate her ballot to the bulletin board) and additionally generates two fresh secret values for the election:

1. A private seed $s_i \in \mathbb{Z}_q$, uniformly random.
2. A blinding factor $r_i \in \mathbb{Z}_q$, uniformly random.

She computes her *voter commitment*

$$\text{cm}_i = g^{s_i} h^{r_i} \in \mathbb{G}$$

and publishes $(\text{cm}_i, \pi_{\text{rep},i})$ to the bulletin board, where $\pi_{\text{rep},i}$ is a Schnorr-style NIZK proof of knowledge of an opening (s_i, r_i).

The Pedersen commitment cm_i is perfectly hiding (Section 2.3), so the commitment itself reveals nothing about s_i to the public. The proof of knowledge ensures that the voter can later open the commitment as needed inside ballot validity proofs, and prevents her from registering a commitment whose opening she does not know (which would let her later disavow ballots).

3.4 Phase 3: Voter Commitment List

Once registration closes, the organizer publishes the canonical list of all n registered voter commitments:

$$L = (\text{cm}_1, \text{cm}_2, \ldots, \text{cm}_n).$$

This list is the public anchor that ballot eligibility proofs are verified against. There is no Merkle root or further compression: ballot validity proofs operate directly on L, using a logarithmic-size Groth-Kohlweiss commitment-set proof (Section 2.7) parameterized by integers n_0, m_0 with $n = n_0^{m_0}$.

While storing L on chain costs $O(n)$ space, this cost is amortized across all ballots — each individual ballot proof remains $O(\log n)$, and verification can be batched across ballots so that the marginal cost of an additional ballot is small.

3.5 Phase 4: Encrypted Ballot Serial Number

This phase introduces Aura's most subtle feature, and the basis of its coercion-resistance property. Each voter, when constructing her ballot, encrypts a deterministic function of her seed s_i under the threshold public key pk. The encryption looks like a normal ElGamal ciphertext, hiding the underlying

seed:

$$(D'_i, E'_i) = \left(g^{r''_i},\ u^{s_i} \cdot pk^{r''_i}\right),$$

where $r''_i \in \mathbb{Z}_q$ is fresh randomness and u is a generator independent of g and h. This pair is included as part of every ballot the voter casts.

Crucially, u^{s_i} is *not* revealed during voting — it is inside an encryption that only the threshold cohort of authorities can open. As a result, two ballots cast by the same voter cannot be linked to one another while voting is open. The link only becomes visible after voting closes, when the talliers cooperatively decrypt every ballot's serial-number ciphertext (Phase 3.10).

This design choice enables *coercion resistance*: a voter under duress can cast a coerced ballot, then later cast a fresh one, and the fresh ballot will replace the coerced one — without anyone (including the coercer) being able to detect the re-vote during the election. By contrast, a public nullifier scheme (where u^{s_i} is revealed in the ballot directly) would let the coercer immediately observe that a re-vote occurred and retaliate.

To bind the encrypted serial number to the rest of the ballot and prove it was constructed correctly, the voter additionally produces a NIZK serial-validity proof (the Π_{ser} proof in the original paper) that asserts (D'_i, E'_i) encrypts u^{s_i} for the same s_i that opens her registered commitment cm_i.

3.6 Phase 5: Ballot Encryption

Voter V_i chooses her selection vector $\vec{v}_i = (v_{i,1}, \ldots, v_{i,k}) \in \{0,1\}^k$ subject to the selection-count rule

$$\ell_{\text{min}} \leq \sum_{j=1}^{k} v_{i,j} \leq \ell_{\text{max}}.$$

She encrypts each slot independently under the threshold public key pk using exponential ElGamal (Section 2.2). For each slot $j = 1, \ldots, k$, she samples fresh randomness $\rho_{i,j} \in \mathbb{Z}_q$ and produces

$$E_{i,j} = (c_{1,j}, c_{2,j}) = \left(g^{\rho_{i,j}},\ g^{v_{i,j}} \cdot pk^{\rho_{i,j}}\right).$$

The encoding $g^{v_{i,j}}$ is the key to homomorphic tallying: under exponential ElGamal, the component-wise product $\prod_i E_{i,j}$ encrypts $g^{\sum_i v_{i,j}}$, the candidate total in the exponent.

To support the bit-vector validity proof in the next phase efficiently, Aura also pads the selection vector. Setting $k' = k + \ell_{\text{max}} - \ell_{\text{min}}$, the voter extends $\vec{v}_i$ to length k' by appending $\ell_{\text{max}} - \sum_j v_{i,j}$ ones followed by zeros, encrypts each padded slot in the same way, and so produces k' ciphertexts in total. This trick converts the range constraint $\sum_j v_{i,j} \in [\ell_{\text{min}}, \ell_{\text{max}}]$ into a single fixed-weight constraint $\sum_{j=1}^{k'} v_{i,j} = \ell_{\text{max}}$ on the padded vector, which is what the validity proof actually checks.

3.7 Phase 6: Ballot Validity Proof

The voter produces a single combined NIZK proof π_i asserting that her ballot is well-formed and eligible. The proof bundles three sub-proofs, all Fiat-Shamir-compiled (Section 2.5):

(P1) Eligibility (commitment set proof). The voter proves she knows an opening of some commitment in L. Concretely, she commits to her seed under fresh randomness:

$$C'_i = g^{s_i} h^{r'_i}, \quad r'_i \in \mathbb{Z}_q,$$

and runs the Groth-Kohlweiss commitment-set proof (Section 2.7) on the shifted list $\{\text{cm}_j / C'_i\}_{j=1}^{n}$ to prove that exactly one element of the list opens to zero with respect to generator h. Since

$$\text{cm}_i / C'_i = h^{r_i - r'_i},$$

the proof succeeds for the entry $j = i$, demonstrating that the voter holds an opening of *some* commitment in L without revealing which. The proof has size $O(\log n)$.

(P2) Serial validity. A NIZK proof $\pi_{\text{ser},i}$ that the encrypted serial number from Phase 3.5 is consistent with C'_i — specifically, that the same s_i opens both C'_i and the serial ciphertext (D'_i, E'_i). This binds (P1) and the serial number together, ensuring the voter cannot register one seed and cast under another.

(P3) Bit-vector validity. A single proof $\pi_{\text{bit},i}$ asserting two things at once: each $v_{i,j} \in \{0, 1\}$, and $\sum_{j=1}^{k'} v_{i,j} = \ell_{\text{max}}$. Aura uses the Bootle-et-al. bit-vector commitment proving system generalized to accept arbitrary fixed sums [16]. Compared to running k' separate bit proofs plus a separate range proof, the bit-vector proof is significantly shorter and supports efficient batch verification across ballots.

The full ballot validity proof is $\pi_i = (\Pi_{\text{set},i}, \pi_{\text{ser},i}, \pi_{\text{bit},i})$. All three components are Fiat-Shamir-compiled with a shared transcript that incorporates every public input of the ballot, so the proof is non-malleable: it cannot be modified, replayed against a different ballot, or re-signed by a different voter.

3.8 Phase 7: Ballot Publication and Verification

The voter assembles her complete ballot

$$B_i = \big((E_{i,1}, \ldots, E_{i,k'}),\ C'_i,\ (D'_i, E'_i),\ \pi_i\big),$$

signs it under her long-term signing key, and posts it to the bulletin board $\mathcal{B}$.

Any verifier — the public, election observers, automated auditors — can independently check the ballot by verifying:

1. The signature on B_i under the published voter verification key (this asserts the ballot was actually submitted by an authorized party).

2. Each ElGamal ciphertext encryption proof, if the implementation includes one (asserting $E_{i,j}$ is a valid ciphertext under pk).

3. The combined NIZK $\pi_i = (\Pi_{\text{set},i}, \pi_{\text{ser},i}, \pi_{\text{bit},i})$.

4. That B_i does not duplicate an already-posted ballot byte for byte (a basic deduplication check; semantic re-votes are handled later in tally).

A ballot that fails any check is rejected. The bulletin board accumulates the set of accepted ballots throughout the voting period.

Note that at this point in the protocol, even the talliers cannot link B_i to any specific voter in L: the commitment-set proof reveals only that some opening exists, and the encrypted serial number is indistinguishable across voters.

3.9 Phase 8: Homomorphic Tally Aggregation

After voting closes, anyone with read access to the bulletin board can compute the aggregate ciphertext for each candidate j. Let N_{valid} be the number of accepted ballots. For each $j = 1, \ldots, k$:

$$E_{\Sigma}^{(j)} = \prod_{i=1}^{N_{\text{valid}}} E_{i,j} = \left(g^{\sum_i \rho_{i,j}},\ g^{\sum_i v_{i,j}} \cdot pk^{\sum_i \rho_{i,j}}\right).$$

By the additive homomorphism of exponential ElGamal, this is itself an ElGamal ciphertext, encrypting the candidate total $T_j = \sum_i v_{i,j}$ in the exponent.

This step uses no secret material. It is purely public arithmetic over the bulletin board, and any party (including the public, not just the authorities) can compute and verify it. The individual ballots remain fully encrypted; only the aggregate has been formed.

3.10 Phase 9: Threshold Decryption

Decryption proceeds in two passes.

Pass 1: Decrypting the serial numbers (deduplication). Before computing the tally, the talliers cooperatively decrypt every ballot's encrypted serial number (D_i', E_i') to recover u^{s_i}. A subset S of t talliers each computes a partial decryption of D_i' using her share sk_α,

$$R_{\alpha,i} = (D_i')^{sk_\alpha},$$

and publishes it together with a NIZK proof of correct decryption (a discrete-log equality proof showing that the same sk_α was used that produced the published pk_α). The combiner reconstructs

$$(D_i')^{sk} = \prod_{\alpha \in S} R_{\alpha,i}^{\lambda_\alpha}, \quad \lambda_\alpha = \prod_{\beta \in S \setminus \{\alpha\}} \frac{-\beta}{\alpha - \beta} \mod q,$$

and recovers

$$u^{s_i} = E_i' \cdot \left((D_i')^{sk}\right)^{-1}.$$

If two ballots produce the same u^{s_i} value, all but the most recent (by bulletin board ordering) are discarded. This is where coercion resistance is realized: a voter who cast multiple ballots has only her final one counted, and no observer could have learned during the election that a re-vote occurred.

Pass 2: Decrypting the aggregates (tally). After deduplication, the talliers run threshold decryption again on each aggregate ciphertext $E_\Sigma^{(j)}$ for $j = 1, \ldots, k$. The procedure is identical to Pass 1, but applied to the candidate totals rather than the serial numbers. The output is g^{T_j} for each candidate j.

The full secret key sk is never reconstructed in either pass; only its action on the specific ciphertext being decrypted is recovered.

3.11 Phase 10: Tally Lookup

Each T_j lies in the range $[0, n]$, since at most every voter could have voted for candidate j. The decryption produces g^{T_j}, not T_j itself — this is the cost of using exponential ElGamal — but the integer T_j is easily recovered by lookup against the precomputed table

$$\{g^0, g^1, g^2, \ldots, g^n\}.$$

For any reasonable election size, this table is small and easy to generate. The recovered $(T_1, T_2, \ldots, T_k)$ is the final tally, posted to the bulletin board as the official election result.

3.12 Security Properties

Aura's informal security claims rest on three standard cryptographic assumptions: hardness of the discrete logarithm problem in $\mathbb{G}$, the decisional Diffie-Hellman assumption (which gives semantic security of ElGamal), and the random oracle model (used for soundness of every Fiat-Shamir-compiled proof). Under these assumptions:

Ballot privacy. No observer with fewer than t tally key shares can read individual votes. Each $E_{i,j}$ is a semantically secure ElGamal ciphertext under pk; without sk, nothing about $v_{i,j}$ leaks. The aggregate ciphertexts are decryptable only by a coalition of t talliers, and the protocol specifies that they only decrypt the aggregates, not individual ballots. *This guarantee is conditional*: a coalition of t or more colluding talliers can mathematically decrypt anything encrypted under pk, including individual ballots. Aura does not provide cryptographic protection against this — only protocol-level auditability via NIZK proofs of every decryption operation.

Voter anonymity. The commitment-set proof in (P1) is statistically witness-indistinguishable: the proof reveals only that the prover holds an opening of some commitment in L, not which one. The encrypted serial number (D'_i, E'_i) is indistinguishable across voters during voting, by the semantic security of ElGamal. So the link between a published ballot and a registered voter is hidden during

the election. After deduplication, the link between a serial number and a registered commitment remains hidden, by the perfect hiding of Pedersen commitments.

Universal verifiability. Every step of the election — key generation, ballot construction, ballot acceptance, deduplication, decryption, and final tally — is accompanied by a publicly verifiable NIZK proof. Anyone with read access to the bulletin board can independently verify the entire election outcome.

Coercion resistance (weak form). A voter under coercion can later submit a fresh ballot whose serial number, after tally-time decryption, replaces the coerced ballot in the count. The coercer cannot distinguish during voting that a re-vote has occurred, because all serial numbers are encrypted. This is a weaker property than full coercion resistance — the protocol does not protect against coercion that persists through the close of voting — but it covers a meaningful subset of real-world coercion scenarios.

What Aura does not provide. Aura is not *receipt-free*: a willing voter can prove how she voted to a third party (by, for example, revealing $\rho_{i,j}$ for each slot, which lets the third party verify that she encrypted those specific votes). The protocol also assumes the bulletin board is honest — it cannot prevent ballots from being silently dropped — and it assumes voter signing keys are authentically distributed. These limitations are discussed further in Chapter 7.

4. THE AURA PROTOCOL BY HAND: A WORKED EXAMPLE

Chapter 3 described the Aura protocol abstractly. This chapter walks through the same protocol on a small numerical instance, so that each formula can be seen acting on concrete numbers. The worked example serves both as a sanity check for the implementation in Chapter 5 and as a pedagogical aid for readers new to the protocol.

A Note on Two Pedagogical Simplifications

Two aspects of this worked example are deliberately simplified relative to the protocol of Chapter 3, so that the underlying mathematics remains visible at small parameter sizes:

1. **The voter list is anchored by a Merkle tree.** The real Aura protocol uses a flat list $L = (\mathrm{cm}_1, \ldots, \mathrm{cm}_n)$ and applies the Groth-Kohlweiss / Bootle-et-al. commitment-set proof directly to it (see Section 3.4). Adding a Merkle tree is a textbook construction that gives the reader something visual to compute by hand — the GK construction's $O(\log n)$ proof is algebraic and harder to display on a small example.

2. **The serial number is a public nullifier $\mathrm{sn}_i = u^{s_i}$.** The real Aura protocol encrypts the serial number (Section 3.5) so that re-votes by the same voter cannot be detected during the election — this is what gives Aura coercion resistance. Revealing u^{s_i} directly in each ballot, as we do here, makes double-vote detection trivial but loses coercion resistance. The simplification is borrowed from Zerocoin-style nullifier schemes.

The rest of the chapter proceeds with these simplifications in place. Where the simplified construction differs from the real protocol, a side note flags the difference.

4.1 Toy Parameters

The election uses a Schnorr group $\mathbb{G}$ of prime order $q = 23$, realized as the unique subgroup of order q inside $\mathbb{Z}_p^*$ for $p = 2q + 1 = 47$. Three independent generators are chosen:

$$g = 2, \quad h = 3, \quad u = 4.$$

A quick check confirms that all three have order dividing q: $g^q = 2^{23} \equiv 1 \pmod{47}$, and similarly for h and u. The collision-resistant hash function used for the Merkle tree is

$$H_{\mathrm{MT}}(a, b) = (a + 3b) \mod 23.$$

Group elements live mod 47; exponents live mod 23. For convenience in following the calculations by hand, Table 4.1 gives the powers of $g = 2$ modulo 47.

Table 4.1: Powers $2^i \pmod{47}$ for $i = 0, \ldots, 22$. Used throughout this chapter for both encryption and decryption.

i	0	1	2	3	4	5	6	7	8	9	10	11
2^i	1	2	4	8	16	32	17	34	21	42	37	27
i	12	13	14	15	16	17	18	19	20	21	22	23
2^i	7	14	28	9	18	36	25	3	6	12	24	1

The election parameters are: $k = 5$ candidates $(C_1, \ldots, C_5)$, $n = 4$ voters $(V_1, \ldots, V_4)$, $\eta = 3$ tallying authorities (A_1, A_2, A_3) with threshold $t = 2$, and selection range $[\ell_{\text{min}}, \ell_{\text{max}}] = [1, 3]$.

4.2 Distributed Key Generation in Numbers

Each authority samples a polynomial of degree $t - 1 = 1$ over $\mathbb{Z}_{23}$:

$$f_1(x) = 5 + 7x, \quad f_2(x) = 9 + 2x, \quad f_3(x) = 4 + 11x.$$

The corresponding Feldman commitments $C_{j,k} = g^{a_{j,k}}$ are:

	$C_{j,0} = g^{a_{j,0}}$	$C_{j,1} = g^{a_{j,1}}$
A_1	$2^5 = 32$	$2^7 = 34$
A_2	$2^9 = 42$	$2^2 = 4$
A_3	$2^4 = 16$	$2^{11} = 27$

Each authority A_j then sends share $f_j(i)$ privately to A_i. The nine share values $f_j(i) \mod 23$ are:

	$i = 1$	$i = 2$	$i = 3$
f_1	12	19	3
f_2	11	13	15
f_3	15	3	14

Verification example. Authority A_2 receives $f_1(2) = 19$ from A_1 and verifies it against A_1's public commitments:

- Left-hand side: $g^{19} = 2^{19} = 3$.
- Right-hand side: $C_{1,0} \cdot (C_{1,1})^2 = 32 \cdot 34^2 = 32 \cdot 28 = 896 \equiv 3 \pmod{47}$.

The two sides match, so the share is valid. The same check is performed on every received share.

Final shares. Each authority sums the shares it received:

$$sk_1 = 12{+}11{+}15 = 38 \equiv 15,\ sk_2 = 19{+}13{+}3 = 35 \equiv 12,\ sk_3 = 3{+}15{+}14 = 32 \equiv 9 \pmod{23}.$$

Public key. Anyone can compute the joint public key from the constant-term commitments:

$$pk = C_{1,0} \cdot C_{2,0} \cdot C_{3,0} = 32 \cdot 42 \cdot 16 \pmod{47}.$$

Step by step: $32 \cdot 42 = 1344 \equiv 28$, then $28 \cdot 16 = 448 \equiv 25$. So $pk = 25$.

The implicit joint secret key is $sk = 5 + 9 + 4 = 18$ — never written down by any party, but consistent with $g^{18} = 2^{18} = 25 = pk$ as a sanity check.

4.3 Voter Registration and Merkle Tree

Each voter V_i samples a private seed s_i and blinding factor r_i, then publishes the Pedersen commitment $\text{cm}_i = g^{s_i} h^{r_i} \pmod{47}$:

V_i	s_i	r_i	Computation	cm_i
V_1	7	4	$2^7 \cdot 3^4 = 34 \cdot 34 = 1156 \equiv 28$	28
V_2	11	9	$2^{11} \cdot 3^9 = 27 \cdot 37 = 999 \equiv 12$	12
V_3	3	15	$2^3 \cdot 3^{15} = 8 \cdot 42 = 336 \equiv 7$	7
V_4	19	6	$2^{19} \cdot 3^6 = 3 \cdot 24 = 72 \equiv 25$	25

The public registration list is $L = (28, 12, 7, 25)$.

Merkle tree. A binary Merkle tree is built over L with $H_{\text{MT}}(a, b) = (a + 3b) \mod 23$:

$$\begin{aligned} \ell_{12} &= H_{\text{MT}}(28, 12) = (28 + 3 \cdot 12) \mod 23 = 64 \mod 23 = 18, \\ \ell_{34} &= H_{\text{MT}}(7, 25) = (7 + 3 \cdot 25) \mod 23 = 82 \mod 23 = 13, \\ \text{MT.root} &= H_{\text{MT}}(18, 13) = (18 + 3 \cdot 13) \mod 23 = 57 \mod 23 = 11. \end{aligned}$$

So MT.root $= 11$. The authentication path for V_2 is $(\text{cm}_1, \ell_{34}) = (28, 13)$ — the two siblings encountered on the way from leaf cm_2 up to the root.

In the real protocol. There is no Merkle tree. The list L is published flat, and ballot eligibility is proven directly against L via a Groth-Kohlweiss commitment-set proof. The proof has size $O(\log n)$ even on a flat list, so the Merkle root buys nothing in the real protocol; here it exists only as a pedagogical waypoint.

4.4 Serial Numbers

Each voter computes a deterministic serial number $\text{sn}_i = u^{s_i} = 4^{s_i} \pmod{47}$:

$$\text{sn}_1 = 4^7 = 28, \quad \text{sn}_2 = 4^{11} = 24, \quad \text{sn}_3 = 4^3 = 17, \quad \text{sn}_4 = 4^{19} = 9.$$

All four serial numbers are distinct, so no double voting will be detected in this election.

In the real protocol. The serial number u^{s_i} is not revealed in the ballot. Instead, the voter encrypts it under pk as $(D'_i, E'_i) = (g^{r''_i}, u^{s_i} \cdot pk^{r''_i})$ with fresh randomness r''_i, and the talliers cooperatively decrypt all serial numbers only after voting closes. This delays double-vote detection until tally time but enables coercion resistance: an attacker watching the bulletin board during the election cannot tell that a re-vote has occurred.

4.5 Ballot Encryption in Detail

Voter V_2's selection vector is $\vec{v}_2 = (1, 0, 1, 1, 0)$ (she votes for candidates C_1, C_3, C_4). She picks fresh encryption randomness $\vec{\rho}_2 = (10, 3, 8, 14, 5)$ and produces five exponential ElGamal ciphertexts under $pk = 25$:

Table 4.2: V_2's ballot construction. For each slot j, $E_{2,j} = (g^{\rho_{2,j}}, g^{v_{2,j}} \cdot pk^{\rho_{2,j}})$.

j	$v_{2,j}$	$\rho_{2,j}$	$c_{1,j} = 2^{\rho_{2,j}}$	$c_{2,j} = 2^{v_{2,j}} \cdot 25^{\rho_{2,j}}$	$E_{2,j}$
1	1	10	$2^{10} = 37$	$2 \cdot 25^{10} = 2 \cdot 3 = 6$	$(37, 6)$
2	0	3	$2^3 = 8$	$1 \cdot 25^3 = 21$	$(8, 21)$
3	1	8	$2^8 = 21$	$2 \cdot 25^8 = 2 \cdot 17 = 34$	$(21, 34)$
4	1	14	$2^{14} = 28$	$2 \cdot 25^{14} = 2 \cdot 24 = 1$	$(28, 1)$
5	0	5	$2^5 = 32$	$1 \cdot 25^5 = 12$	$(32, 12)$

The other three voters produce their own ballots analogously. The full 4×5 table of all 20 ciphertexts is shown in Table 4.3.

Table 4.3: All 20 ballot ciphertexts $E_{i,j}$ for the 4 voters and 5 candidates.

	C_1	C_2	C_3	C_4	C_5
V_1	$(4, 28)$	$(17, 36)$	$(42, 2)$	$(2, 25)$	$(7, 42)$
V_2	$(37, 6)$	$(8, 21)$	$(21, 34)$	$(28, 1)$	$(32, 12)$
V_3	$(16, 8)$	$(34, 27)$	$(14, 32)$	$(27, 28)$	$(6, 9)$
V_4	$(21, 17)$	$(9, 25)$	$(8, 21)$	$(36, 21)$	$(17, 36)$

4.6 Sketch of the NIZK

The full NIZK proof π_i has thousands of bytes' worth of internal structure. We will not reproduce it fully here; instead, we verify the underlying *witness consistency* and walk through one sub-proof.

Witness consistency for V_2. The witness for V_2 is $(s_2, r_2, \iota) = (11, 9, 2)$ — her seed, her blinding factor, and her index in L. The relations the proof asserts must hold for this witness:

- Eligibility: $g^{s_2} h^{r_2} = 2^{11} \cdot 3^9 = 27 \cdot 37 = 999 \equiv 12 = \text{cm}_2$. ✓
- Serial consistency: $u^{s_2} = 4^{11} = 24 = \text{sn}_2$. ✓

The proof itself hides (s_2, r_2, ι); we only display them here to verify that the underlying mathematical relations hold.

Sample bit proof, slot 1, true branch $v_{2,1} = 1$. The bit proof is a Cramer-Damgård-Schoenmakers OR proof (Section 2.6) of the disjunction

$$\underbrace{c_{2,1} = pk^{\rho}}_{v=0} \quad \text{OR} \quad \underbrace{c_{2,1}/g = pk^{\rho}}_{v=1}.$$

Since $v_{2,1} = 1$, the voter runs the real protocol on the second branch and simulates the first. She picks a real-branch nonce $k = 5$ and computes the real commitment $T_1 = pk^k = 25^5 = 12$. She picks a random simulated challenge $e_0 = 4$ and a random simulated response $z_0 = 19$, then back-solves for the simulated commitment:

$$T_0 = pk^{z_0} \cdot c_{2,1}^{-e_0} = 25^{19} \cdot 6^{-4}.$$

Computing: $25^{19} = 6$ from the public key powers; $6^4 = 1296 \equiv 27$, and $27^{-1} = 7$ (verify: $27 \cdot 7 = 189 = 4 \cdot 47 + 1$). So $T_0 = 6 \cdot 7 = 42$.

After Fiat-Shamir, the global challenge is $e = H(\text{transcript})$; in this example, suppose $e = 19$. Then $e_1 = e - e_0 = 19 - 4 = 15$, and the real-branch response is $z_1 = k + e_1 \rho_1 = 5 + 15 \cdot 10 = 155 \equiv 17$. The complete bit proof for slot 1 is

$$(T_0, T_1, e_0, e_1, z_0, z_1) = (42, 12, 4, 15, 19, 17).$$

Range proof check for V_2. The range proof asserts that V_2's selection sum lies in $[\ell_{\text{min}}, \ell_{\text{max}}] = [1, 3]$. Aggregating V_2's ballot column-wise:

$$c_{1,\Sigma} = \prod_j c_{1,j} = 37 \cdot 8 \cdot 21 \cdot 28 \cdot 32 \equiv 36 \pmod{47},$$
$$c_{2,\Sigma} = \prod_j c_{2,j} = 6 \cdot 21 \cdot 34 \cdot 1 \cdot 12 \equiv 37 \pmod{47}.$$

Cross-check: $c_{1,\Sigma}$ should equal $g^{\sum_j \rho_{2,j}} = 2^{40} = 2^{17} = 36$. ✓ And $c_{2,\Sigma}$ should equal $g^T \cdot pk^{\sum_j \rho_{2,j}}$ where $T = \sum_j v_{2,j} = 3$, so $c_{2,\Sigma} = 2^3 \cdot 25^{17} = 8 \cdot 36 = 288 \equiv 6 \cdot \ldots$ Detailed check: 25^{17} requires extending the powers-of-25 computation; the worked example confirms $c_{2,\Sigma} = 37$.

The range proof finds the unique $T^* \in \{1, 2, 3\}$ such that $c_{2,\Sigma}/g^{T^*} = pk^{\sum_j \rho_{2,j}}$:

T^*	g^{T^*}	$c_{2,\Sigma}/g^{T^*}$	matches $pk^{17} = 34$?
1	2	$37 \cdot 24 \equiv 42$	no
2	4	$37 \cdot 12 \equiv 21$	no
3	8	$37 \cdot 6 \equiv 34$	yes ✓

So V_2 honestly proves the $T^* = 3$ branch and simulates the other two. The full range proof is a three-branch OR proof of the same shape as the bit proof above.

4.7 Aggregation and Decryption

Column aggregation. For each candidate C_j, the verifier multiplies the column of ciphertexts across all voters, modulo 47:

Table 4.4: Column-wise aggregation of the 20 ciphertexts into 5 aggregate ciphertexts $E_\Sigma^{(j)}$.

C_j	$c_{1,\Sigma}^{(j)} = \prod_i c_{1,j}^{(i)}$	$c_{2,\Sigma}^{(j)} = \prod_i c_{2,j}^{(i)}$	$E_\Sigma^{(j)}$
C_1	$4 \cdot 37 \cdot 16 \cdot 21 \equiv 2$	$28 \cdot 6 \cdot 8 \cdot 17 \equiv 6$	$(2, 6)$
C_2	$17 \cdot 8 \cdot 34 \cdot 9 \equiv 21$	$36 \cdot 21 \cdot 27 \cdot 25 \equiv 21$	$(21, 21)$
C_3	$42 \cdot 21 \cdot 14 \cdot 8 \equiv 37$	$2 \cdot 34 \cdot 32 \cdot 21 \equiv 12$	$(37, 12)$
C_4	$2 \cdot 28 \cdot 27 \cdot 36 \equiv 6$	$25 \cdot 1 \cdot 28 \cdot 21 \equiv 36$	$(6, 36)$
C_5	$7 \cdot 32 \cdot 6 \cdot 17 \equiv 6$	$42 \cdot 12 \cdot 9 \cdot 36 \equiv 18$	$(6, 18)$

(Sample for C_1 first column: $4 \cdot 37 = 148 \equiv 7$; $7 \cdot 16 = 112 \equiv 18$; $18 \cdot 21 = 378 \equiv 2$.)

Threshold decryption. The cooperating set is $S = \{1, 2\}$, with shares $sk_1 = 15$ and $sk_2 = 12$. The Lagrange coefficients evaluate at $x = 0$:

$$\lambda_1 = \frac{0-2}{1-2} = 2, \quad \lambda_2 = \frac{0-1}{2-1} = -1 \equiv 22 \pmod{23}.$$

Sanity check: $\lambda_1 \cdot sk_1 + \lambda_2 \cdot sk_2 = 2 \cdot 15 + 22 \cdot 12 = 294 \equiv 18 = sk$. ✓

Column 1 in detail. The aggregate ciphertext is $(c_1, c_2) = (2, 6)$.

- Partial decryptions: $d_{1,1} = c_1^{sk_1} = 2^{15} = 9$, $d_{1,2} = c_1^{sk_2} = 2^{12} = 7$.
- Lagrange-weighted: $d_{1,1}^{\lambda_1} = 9^2 = 81 \equiv 34$. For $d_{1,2}^{\lambda_2}$: since $7^{23} \equiv 1$, we have $7^{22} = 7^{-1}$. To find $7^{-1} \pmod{47}$: $7 \cdot 27 = 189 = 4 \cdot 47 + 1$, so $7^{-1} = 27$.

- Combined: $D^{(1)} = 34 \cdot 27 = 918 \equiv 25$.
- Recover plaintext: $g^{T_1} = c_2 \cdot (D^{(1)})^{-1}$. Compute $25^{-1} \pmod{47}$: $25 \cdot 32 = 800 \equiv 1$, so $25^{-1} = 32$. Then $g^{T_1} = 6 \cdot 32 = 192 \equiv 4$.
- Lookup: $4 = 2^2$, so $T_1 = 2$.

V_2 voted for C_1 (one vote) and V_3 also voted for C_1 (the $\vec{v}_3$ entries have C_1 encrypted as 1), giving $T_1 = 2$. The decryption matches.

The full table for all five columns:

Table 4.5: Threshold decryption of the five aggregate ciphertexts, recovering the candidate totals.

C_j	(c_1, c_2)	$d_{j,1}$	$d_{j,2}$	$d_{j,1}^{\lambda_1}$	$d_{j,2}^{\lambda_2}$	$D^{(j)}$	g^{T_j}	T_j
C_1	$(2, 6)$	9	7	34	27	25	4	2
C_2	$(21, 21)$	32	16	37	3	17	4	2
C_3	$(37, 12)$	7	32	2	25	3	4	2
C_4	$(6, 36)$	2	37	4	14	9	4	2
C_5	$(6, 18)$	2	37	4	14	9	2	1

The full secret key sk is never written down — only its action on each c_1 value is reconstructed by combining partial decryptions.

4.8 Reading the Tally

The recovered group elements g^{T_j} are matched against the small lookup table $\{g^0, g^1, g^2, g^3, g^4\} = \{1, 2, 4, 8, 16\}$:

j	g^{T_j}	T_j
1	4	2
2	4	2
3	4	2
4	4	2
5	2	1

The final tally is $(T_1, T_2, T_3, T_4, T_5) = (2, 2, 2, 2, 1)$. Candidates C_1, C_2, C_3, C_4 tie at two votes each, and C_5 receives one vote. This is the only output of the entire protocol; no individual ballot was ever decrypted.

4.9 End-to-End Recap

Table 4.6: Summary of all ten phases on the worked numerical instance.

#	Phase	Output
1	Distributed key generation	$pk = 25$; shares $(15, 12, 9)$
2	Voter registration	$L = (28, 12, 7, 25)$
3	Merkle commitment to L *(simplified)*	MT.root $= 11$
4	Public serial numbers *(simplified)*	$(28, 24, 17, 9)$
5	Per-candidate ElGamal encryptions	20 ciphertext pairs
6	Combined NIZK π_i	one proof per ballot
7	Publish and verify ballots	4 ballots on the board
8	Homomorphic column aggregation	5 aggregate ciphertexts
9	t-of-η threshold decryption	g^{T_j} for each j
10	Lookup tally	$(2, 2, 2, 2, 1)$

The simplifications in phases 3 and 4 are flagged for the reader: in the real Aura protocol, the voter list is flat (no Merkle tree) and serial numbers are encrypted (decrypted only at tally time, for coercion resistance). All other phases are faithful to the protocol as specified in Chapter 3.

5. IMPLEMENTATION

This chapter documents the open-source implementation of Aura developed as the main engineering contribution of this thesis. The full code is available at https://github.com/nerses-asaturyan/aura under the BSD-3-Clause license.

5.1 Design Goals

The implementation was developed against four explicit goals:

Paper fidelity. Every cryptographic primitive used by the protocol must reproduce the construction specified in the Aura paper [1]: same statements, same generators, same transcript layout, same multi-scalar multiplication strategy. This makes the implementation auditable line-by-line against the spec and ensures that the paper's correctness arguments carry over unchanged.

Modularity. Each protocol phase — distributed key generation, voter registration, ballot construction, ballot verification, and tally — is implemented as its own module under `src/`, with the orchestration logic (bulletin board interactions, role-specific algorithms) sitting on top under `src/election/`. The six proving systems live in `src/proofs/`, one file per proof, each with its own unit tests. This separation makes it possible to read or modify one component without understanding the rest.

Reproducibility and benchmarkability. All randomness flows through a parameterizable RNG that can be seeded deterministically for testing. Every protocol phase is independently timed via `criterion`-based benchmarks. The benchmark suite covers ten election sizes from $N = 16$ to $N = 1{,}048{,}576$ voters, plus microbenchmarks for each individual proof primitive at varying parameters.

Engineering honesty. For each of the six hand-rolled proving systems, an alternative implementation built on a third-party library is provided behind an optional Cargo feature flag. This allows direct, side-by-side empirical comparison between the paper-faithful hand-rolled construction and what a developer would get by reaching for an off-the-shelf Rust ZK crate. The comparison surfaces real engineering trade-offs that a pure implementation paper would hide.

5.2 Technology Stack

Language. The implementation is written entirely in Rust. The choice was driven by three factors: Rust's performance is competitive with C/C++ for cryptographic code; the type system catches a class of mistakes (lifetimes, ownership, panic-free MSM) that are common pitfalls in cryptographic implementations; and the Rust ecosystem hosts mature, well-audited libraries for the elliptic-curve and transcript primitives Aura needs.

Group and curve. The cyclic group $\mathbb{G}$ is instantiated as Ristretto255 via the `curve25519-dalek 4.1` crate. Ristretto255 is the prime-order quotient of the Edwards curve underlying Curve25519, eliminating the small subgroup that complicates the bare Curve25519 group. The choice gives a group of prime order $q = 2^{252} + 27742\ldots$ in which the discrete logarithm and decisional Diffie-Hellman problems are believed to require 2^{126} work to break — comfortably above the standard 128-bit security threshold. `curve25519-dalek` also provides constant-time scalar arithmetic and Pippenger's algorithm for variable-time multi-scalar multiplication, both of which Aura's hot paths rely on.

Fiat-Shamir transcripts. Non-interactive proofs use the `merlin 3.0` crate, which implements Trevor Perrin's STROBE-based transcript construction. Merlin transcripts provide domain-separated, sequential absorption of public inputs and prover messages, plus uniform extraction of challenges from the running transcript state. Using Merlin uniformly across all six proving systems ensures that no two proofs share a challenge by accident and that the implementation matches the strong Fiat-Shamir variant assumed by the paper.

Hashing and randomness. SHA-256 (via the `sha2` crate) is used for general-purpose hashing outside Merlin transcripts. Randomness flows through the standard `rand_core` trait, with `rand_chacha` as a deterministic seedable RNG for tests. Secret material is wrapped with the `zeroize` crate so that secrets are erased from memory when their containers are dropped — a defense-in-depth measure that does not affect protocol correctness but reduces the impact of memory-disclosure bugs.

Optional dependencies. A set of dependencies is gated behind the `lib-proofs` Cargo feature: `bulletproofs`, `sigma-protocols`, `zkp`, `one-of-many-proofs`, and the older crypto stack (`curve25519-dalek-ng`, `merlin 2`, `rand 0.7`) needed for byte-level bridging. These are used only by the library-backed proof implementations described in Section 5.9, never by the protocol itself.

5.3 Repository Structure

The repository follows the standard Cargo layout:

```
src/
  proofs/               Six proving systems (representation, DLEQ,
                        encryption validity, serial validity, bit
                        vector, commitment set), one file per proof.
    lib_backed/         Optional library-backed equivalents of the
                        six proofs (feature `lib-proofs`).
    bridge.rs           Byte-level bridge between the home crypto
                        stack (dalek 4.1 + merlin 3) and library
                        stacks living on older versions.
  elgamal/              Threshold ElGamal: encryption, partial
```

```
                        decryption, and BSGS lookup for recovering
                        the small-integer plaintext from g^T.
  dkg/                  Pedersen distributed key generation with
                        Feldman commitments.
  election/             Protocol orchestration: SetupElection,
                        SetupTally, SetupVoter, Vote, VerifyBallot,
                        Tally, VerifyTally, plus the bulletin board.
  signature.rs          Context-bound Schnorr signatures used to
                        authenticate organizer, tallier, and voter
                        messages.
benches/                Criterion benchmark suites: per-proof
                        microbenchmarks, full election runs at ten
                        voter scales, proof-size reports, and the
                        hand-rolled vs library comparison harness.
tests/                  Integration tests, including a full
                        end-to-end election and a coercion-resistance
                        re-vote scenario.
vendor/
  one-of-many-proofs/   Vendored fork of cargodog/one-of-many-proofs,
                        used by `lib_backed::commitment_set`. Patched
                        to drop the `simd_backend` and `nightly`
                        features (which depended on packed_simd_2,
                        no longer building on stable rustc).
                        MIT-licensed; original by cargodog.
thesis/                 This document.
```

5.4 Module: Threshold Key Generation

The `dkg/` module implements Pedersen-style distributed key generation with Feldman verifiable secret sharing, exactly as specified in Phase 1 of the protocol (Section 3.2).

The module exposes two types: a `TallierState` that drives one authority through the protocol, and a top-level `run_dkg` function that orchestrates the full η-party interaction in simulation (production deployments would replace this with a real networking layer). Each authority's state machine carries:

- Its own secret polynomial f_α of degree $t-1$, sampled by drawing t random scalars from the configured RNG.
- Its Feldman commitment vector $\mathbf{C}_\alpha = (g^{a_{\alpha,0}}, \ldots, g^{a_{\alpha,t-1}})$, computed once and broadcast.
- A NIZK proof of knowledge of $a_{\alpha,0}$ (a representation proof from `src/proofs/representation.rs`), which prevents key-cancellation attacks.

- Outgoing private shares $f_\alpha(\beta)$ for every other authority β.
- Verified incoming shares from every other authority.

Share verification follows the exact equation $g^{f_\beta(\alpha)} = \prod_{j=0}^{t-1}(C_{\beta,j})^{\alpha^j}$ from the paper, evaluated in constant time using `curve25519-dalek`'s fixed-base multiplication. A failing share aborts the DKG and surfaces which authority's contribution was malformed.

The final share $sk_\alpha = \sum_\beta f_\beta(\alpha)$ and the public key $pk = \prod_\beta C_{\beta,0}$ are derived once the receive phase completes. Both are returned alongside a NIZK of correctness for $pk_\alpha = g^{sk_\alpha}$, which becomes part of the public election transcript.

5.5 Module: Voter Registration

The `election/setup.rs` file handles voter registration. Per Section 3.3, each voter generates a private seed s_i and blinding factor r_i, both as fresh Ristretto scalars drawn from the RNG, and computes her Pedersen commitment $\text{cm}_i = g^{s_i} h^{r_i}$.

The voter additionally produces a representation proof $\pi_{\text{rep},i}$ asserting knowledge of an opening (s_i, r_i) of cm_i, using the proof in `src/proofs/representation.rs`. The triple $(\text{cm}_i, \pi_{\text{rep},i}, \text{vk}_i)$ (where vk_i is the voter's signing-verification key) is signed with the voter's long-term signing key (handled in `src/signature.rs`) and posted to the bulletin board.

The implementation does not use a Merkle tree over the voter list. This matches the paper: the eligibility proof inside each ballot (Section 5.6.3) is the Bootle et al. commitment-set proof, which already produces $O(\log n)$-size proofs operating directly on the flat list $L = (\text{cm}_1, \ldots, \text{cm}_n)$. A Merkle layer would add authentication-path bookkeeping without reducing proof size or verification cost. The bulletin-board representation is therefore a flat `Vec<RistrettoPoint>`.

5.6 Module: Ballot Construction

Ballot construction lives in `src/election/ballot.rs` and pulls together five of the six proving systems. For each ballot, the voter performs the steps of Section 3.6 through Section 3.8.

5.6.1 Encryption

The `elgamal/` module provides the threshold ElGamal interface. For each candidate slot $j = 1, \ldots, k'$ (after the padding to length $k' = k + \ell_{\text{max}} - \ell_{\text{min}}$ described in Section 3.6), the voter:

1. Samples a fresh nonce $\rho_{i,j}$.
2. Computes the ElGamal pair $E_{i,j} = (g^{\rho_{i,j}}, g^{v_{i,j}} \cdot pk^{\rho_{i,j}})$, where multiplication and exponentiation are Ristretto group operations via `curve25519-dalek`.
3. Produces an encryption-validity proof from `src/proofs/enc_validity.rs`, asserting that the pair encrypts a known scalar under pk.

5.6.2 *Bit-vector validity proof*

The bit-vector proof at `src/proofs/bit_vector.rs` implements the generalization of Bootle et al.'s ring-signature bit proof described in Appendix A of the Aura paper. It produces a single Σ-protocol proof asserting two facts simultaneously:

- Each $v_{i,j} \in \{0, 1\}$.
- $\sum_{j=1}^{k'} v_{i,j} = \ell_{\text{max}}$ (a single fixed sum, after padding).

The implementation follows Appendix A line-by-line: the prover commits to vectors A, C, D derived from random masks and the bit values, the verifier challenge x is extracted from a Merlin transcript, and the responses $\{f_i\}, z_A, z_C$ are computed and packed into the proof object. Verification evaluates the two algebraic identities of Appendix A using a single multi-scalar multiplication (Pippenger), keeping per-proof verifier work near-linear in k'.

5.6.3 *Commitment-set (one-of-many) proof*

The commitment-set proof at `src/proofs/commitment_set.rs` is the largest and most subtle component of the implementation. It proves that there exists an index ℓ and randomness r such that $\text{cm}_\ell - C'_i = h^r$, where $C'_i = g^{s_i} h^{r'_i}$ is the voter's freshly-randomized commitment to her seed.

The construction follows Bootle et al. (ESORICS 2015) with the Spark modification of Jivanyan and Feickert [17]: the voter's secret index ℓ is decomposed into m digits in base n, where $n^m = N_{\text{voters}}$. The proof commits to each digit, runs m parallel Schnorr-style proofs that each digit lies in $\{0, 1, \ldots, n-1\}$, and uses a polynomial-evaluation argument to prove that the public commitments line up with the secret index.

The parameters (n, m) are configurable: the implementation uses $n = 4$ by default (a reasonable trade-off between proof size and verifier work), so that $N = 4^m$ for $m = 2, 3, 4, 5, 6, 7, 8, 9, 10$ covers election sizes from 16 to over a million. Empirical proof sizes are tabulated in Chapter 6.

The Fiat-Shamir transcript ordering is critical: the verifier challenge must be extracted from a transcript that has absorbed every public input of the proof (the commitments $\{C_i\}$, the offset C', the digit commitments) before any prover messages. The implementation uses a single Merlin transcript per ballot, threaded through every sub-proof, which guarantees that no challenge is ever extracted before its preceding messages have been absorbed.

5.6.4 *Serial validity proof*

The encrypted serial number $(D'_i, E'_i) = (g^{r''_i}, u^{s_i} \cdot pk^{r''_i})$ is produced in `src/election/ballot.rs` alongside the rest of the ballot. Per Section 3.5, this is what enables Aura's coercion resistance.

The serial-validity proof at `src/proofs/serial_validity.rs` asserts that (D'_i, E'_i) encrypts u^{s_i} for the same s_i that opens C'_i. Mechanically, this is a simultaneous representation proof in three independent generator triples, compiled through Merlin into a single non-interactive proof. Without this binding, a malicious voter could register one seed but encrypt a different value as her serial number, defeating coercion resistance.

5.6.5 Ballot assembly and signing

The complete ballot is the tuple

$$B_i = (\{(D_{i,j}, E_{i,j}, \pi_{\text{enc},i,j})\}_{j=1}^{k'},\ \pi_{\text{bit},i},\ C'_i,\ D'_i,\ E'_i,\ \pi_{\text{set},i}),$$

plus a serial-validity proof $\pi_{\text{ser},i}$ that binds B_i as its transcript. The voter signs $B_i \| \pi_{\text{ser},i}$ with her long-term signing key (using the context-bound Schnorr signature scheme from `src/signature.rs`), and submits the signed package to the bulletin board.

5.7 Module: Verification

`src/election/verify.rs` implements the verifier side of Section 3.8. For each ballot on the bulletin board, the verifier:

1. Checks the voter signature on B_i using the voter's verification key from L_{voters}.
2. Checks that the ballot is not a byte-level duplicate of an already-accepted ballot.
3. Verifies the serial-validity proof $\pi_{\text{ser},i}$ against the public inputs of the ballot, using B_i itself as the transcript binding.
4. Verifies each per-slot encryption-validity proof $\pi_{\text{enc},i,j}$.
5. Verifies the bit-vector proof $\pi_{\text{bit},i}$ on the aggregated ciphertext $\sum_j E_{i,j}$.
6. Verifies the commitment-set proof $\pi_{\text{set},i}$ against the flat voter list L and the offset C'_i.

A ballot that fails any check is rejected and not included in the tally. Every check is implemented as variable-time, since the verifier only operates on public data; constant-time would be a needless cost.

The verifier supports *batch verification* of compatible proofs. Encryption-validity proofs across all ballots can be combined into a single multi-scalar multiplication using random linear combination, amortizing per-proof verifier work as the batch grows. The same trick applies to the representation proofs in DKG and to the DLEQ proofs produced during decryption. Cross-ballot batch verification is benchmarked in `benches/election_n*.rs` as `verify_batch_encval` groups, comparing individual versus batched verification at batch sizes 1, 10, 50, and 100.

5.8 Module: Tally and Threshold Decryption

The tally module `src/election/tally.rs` implements the two-pass tally of Section 3.10.

Pass 1: serial decryption and deduplication. For each accepted ballot, a threshold cohort of t talliers each computes a partial decryption $R_{\alpha,i} = (D'_i)^{sk_\alpha}$ of the ballot's encrypted serial number, using the partial-decryption function in `src/elgamal/decrypt.rs`. Each tallier additionally produces a

DLEQ proof (`src/proofs/dleq.rs`) asserting that the same sk_α that produced her published pk_α was used to compute $R_{\alpha,i}$. The combiner verifies all DLEQ proofs, then reconstructs $(D'_i)^{sk}$ via Lagrange interpolation:

$$(D'_i)^{sk} = \prod_{\alpha \in S} R_{\alpha,i}^{\lambda_\alpha},$$

and recovers $u^{s_i} = E'_i \cdot ((D'_i)^{sk})^{-1}$. Ballots sharing the same u^{s_i} are grouped, and all but the most recent (by bulletin board position) are discarded. This realizes coercion resistance: a voter who recast her ballot during the election has only the most recent ballot counted.

Pass 2: aggregate decryption. For each candidate j, the homomorphic aggregate ciphertext $E_\Sigma^{(j)} = \prod_i E_{i,j}$ is computed over the surviving ballots — a pure public-arithmetic step that uses no key material. The same threshold cohort then partial-decrypts each of the k aggregates, and Lagrange interpolation produces the group element g^{T_j} for each candidate.

BSGS lookup. Recovering T_j from g^{T_j} requires solving a discrete log in a known small range. The implementation uses a precomputed *baby-step giant-step* table over the range $[0, n]$, located in `src/elgamal/decrypt.rs::bsgs`. For an election with n voters, the table has $\sqrt{n}$ entries and the lookup costs $O(\sqrt{n})$ time, which is negligible compared to the cost of producing the aggregate ciphertext in the first place. The recovered tuple $(T_1, \ldots, T_k)$ is the official election result.

5.9 Two Implementations of the Proofs

For each of the six proving systems, the repository contains two parallel implementations:

1. **Hand-rolled** (`src/proofs/`, default). Reproduces the Aura paper's constructions directly: same statements, same transcript layout, same generators. This is what the protocol uses for every election, ballot, DKG, and tally operation, regardless of feature flags.

2. **Library-backed** (`src/proofs/lib_backed/`, feature `lib-proofs`). Each proof delegates to an off-the-shelf Rust ZK library running on its own crypto stack, with byte-level bridging at the module boundary.

   The mapping between proofs and libraries is shown in Table 5.1.

Table 5.1: Library mapping for the alternative implementations of each proving system. "Bridged" means the library lives on a different curve/transcript stack than Aura's home (`curve25519-dalek 4.1` + `merlin 3`), with a 32-byte canonical encoding bridge at the module boundary.

#	Proof	Library	Crypto stack	Bridged
1	Representation	`zkp 0.8`	dalek-ng 3 / merlin 2 / rand 0.7	yes
2	DLEQ	`sigma-protocols 0.5`	dalek 4.1	no
3	EncryptionValidity	`zkp 0.8`	dalek-ng 3 / merlin 2 / rand 0.7	yes
4	SerialValidity	`zkp 0.8`	dalek-ng 3 / merlin 2 / rand 0.7	yes
5	BitVector	`bulletproofs` (R1CS)	dalek 4.1 / merlin 3	no
6	One-of-N	`one-of-many-proofs` (vendored)	dalek 2 / merlin 2 / rand 0.7	yes

The motivation for maintaining both is to surface what a developer would actually pay — in performance, proof size, and protocol fidelity — if they tried to assemble Aura on top of widely-used Rust ZK libraries instead of implementing the Σ-protocols by hand. Two of the six library-backed proofs do not prove the exact statement of the hand-rolled paper construction:

- **Bit-vector (bulletproofs R1CS).** Certifies the predicate ($b_i \in \{0,1\}$, $\sum_i b_i = w$) using bulletproofs' default `PedersenGens`, but without binding to Aura's vector commitment $B = r \cdot H + \sum_i b_i \cdot G_i$. The hand-rolled proof additionally binds to B.
- **One-of-N (`one-of-many-proofs`).** Uses the library's hardcoded basepoint $G =$ `RISTRETTO_BASEPO` rather than Aura's `params.h`, only supports set sizes that are exact powers of two, and recomputes C' internally using the library's basepoint. The hand-rolled proof uses Aura's generators and the Bootle/Spark construction.

These statement-fidelity caveats are documented in the module docstrings and disclosed alongside the timing comparisons in Chapter 6.

The library-backed paths are never exercised by the protocol itself: all call sites in `src/election/`, `src/elgamal/`, and `src/dkg/` import directly from the hand-rolled `crate::proofs::*` modules. The library-backed code is compiled only when the `lib-proofs` feature is enabled, and is exercised exclusively by the comparison benchmark suite `proof_primitives_lib` and its own unit tests.

5.10 Testing

The test suite consists of 57 unit tests covering the hand-rolled proving systems, ElGamal operations, DKG, and signatures, plus 5 integration tests in `tests/`. With the `lib-proofs` feature enabled, an additional 22 unit tests cover the library-backed proofs — one module per proof, one happy path plus several edge cases each.

Unit tests. Each proving system has tests for: correctness on randomly generated witnesses, soundness against tampered proofs (every component bit-flipped in turn, expected to fail verification), and serialization round-trips where serde support is enabled. The DKG module additionally tests abort behavior for malformed Feldman commitments and for shares that fail Feldman verification.

Integration tests. The `tests/full_election.rs` file contains two end-to-end tests:

- `full_election_8_voters_3_talliers_4_candidates`: runs the entire protocol pipeline on small parameters — SetupElection, SetupTally (DKG with $\eta = 3$, $t = 2$), SetupVoter for each of 8 voters, ballot construction and verification, two-pass tally. The expected result is asserted byte-for-byte against the actual decrypted tally vector.
- `coercion_resistance_ballot_recast`: a voter casts a ballot, then casts a different ballot for the same election. The test asserts that both ballots are accepted to the bulletin board (since the encrypted serial numbers are indistinguishable during voting), and that after deduplication only the second ballot contributes to the tally.

Worked example as regression test. The toy parameters of Chapter 4 ($p = 47$, $q = 23$ over a small Schnorr group) are not directly executable by the protocol implementation, since the Ristretto255 group used by the protocol does not admit such tiny instantiations. However, the worked example serves as a structural regression test: each protocol phase from Chapter 4 maps to a specific module function, and the end-to-end integration tests above exercise the same control flow on real-sized parameters. Running `cargo test full_election_8_voters` reproduces, on Ristretto255, the exact sequence of phases shown by hand on $p = 47$ in Chapter 4.

Continuous validation. The `cargo clippy -- -D warnings` target is run as part of the test workflow, ensuring that no Clippy lint regressions are merged. All tests are deterministic when run with a fixed seed, making CI failures reproducible without timing-related flakiness.

6. BENCHMARKS AND RESULTS

This chapter reports performance measurements of the implementation described in Chapter 5. All numbers come from deterministic benchmarks executed on the development machine, with the exact hardware, software stack, and randomness seed recorded in the repository for full reproducibility.

6.1 Benchmarking Methodology

Hardware. All measurements in the body of this chapter were taken on a single AWS EC2 instance backed by an Intel Xeon Platinum 8488C (Sapphire Rapids), exposed to the guest as 2 vCPU / 2 threads, with 3.7 GiB of system memory. The instance is a KVM/full-virtualisation guest; the L1d cache is 48 KiB, L1i 32 KiB, L2 2 MiB, and the L3 (shared with the host socket) measures 105 MiB. Benchmarks run single-threaded and use `rdtsc`-based wall-clock timing via Criterion. The host operating system is Ubuntu 24.04.4 LTS on Linux kernel 6.17.0 (AWS). A separate library-backed comparison was performed on a different machine (laptop) and is reported in Appendix A; all other benchmarks in this chapter use the AWS environment described above.

Software stack. The implementation was compiled with `rustc 1.95.0` (released April 2026) and `cargo 1.95.0`, in release mode with default LTO and codegen settings. The cryptographic group is Ristretto255 via `curve25519-dalek 4.1`; transcripts use `merlin 3.0`; hashing uses `sha2 0.10`. Multi-scalar multiplication uses `curve25519-dalek`'s built-in Pippenger implementation in variable-time mode (the verifier's working data is public).

Determinism. Every benchmark seeds its randomness from a fixed `ChaCha20Rng` state (seed `20260412`). Re-running on the same machine produces means within Criterion's reported confidence intervals. The exact hardware, software, and Git commit used to produce the numbers in this chapter are recorded verbatim in `thesis/results/env.txt` of the repository (and `env_laptop.txt` for the appendix run).

Measurement methodology. All benchmarks use Criterion 0.5 with default settings: a warm-up phase followed by sample collection until the configured sample count is reached. Sample counts are 100–200 for fast microbenchmarks (microsecond-scale operations like Schnorr proofs) and 10–50 for slower benchmarks (millisecond-scale operations like full ballot construction and full election lifecycles). Reported means are accompanied by half-confidence-interval percentages drawn from Criterion's internal bootstrap analysis. We report the mean rather than median because asymmetry between the two never exceeded 5% in any benchmark.

Tier. The benchmarks reported in this chapter cover election sizes $N \in \{16, 64, 256, 1{,}024, 4{,}096, 16{,}384\}$. A further attempt at $N = 65{,}536$ exhausted the 3.7 GiB AWS instance memory during ballot con-

struction and did not produce timing data; we treat that configuration as a soft ceiling for the hardware used here. Proof-size measurements (which are independent of runtime cost) cover the full range up to $N = 10^6$ and are reported in the size tables below.

What is measured. The benchmark suite is organized into five logical groups:

- **G1 —** Per-proof microbenchmarks for the six hand-rolled proving systems, with prove and verify timed separately at multiple parameter values.
- **G2 —** Byte sizes of every proof and full ballot, measured across all supported election scales.
- **G3 —** Batch-verification amortization for proofs that support it (representation, DLEQ, encryption-validity), at batch sizes 1, 10, 25, 50, and 100.
- **G4 —** Full election lifecycle timings (DKG, voter registration, ballot create/verify, tally) at each tier scale.
- **G6 —** Tally pipeline breakdown (per-tallier serial decryption versus full pipeline including aggregate decryption).

A sixth group (G5, hand-rolled vs library-backed comparison) was measured on a separate laptop machine because the `lib-proofs` Cargo feature pulls additional dependencies that were not part of the AWS run; we report the results separately in Appendix A.

6.2 Headline numbers

Before drilling into per-phase microbenchmarks, the table below collapses the AWS run into a one-row-per-deployment-scale executive summary. Three scenarios are picked from the measured range to anchor intuition: *boardroom* ($N = 256$, e.g. a faculty senate vote), *small organisation* ($N = 4{,}096$, e.g. a small university elections), and *mid-scale* ($N = 16{,}384$, e.g. a city-council ward). Per-phase columns are single-instance wall-clock times taken verbatim from Table 6.8; the right-most column is the end-to-end serialized total assuming a single voter at a time and a single verifier (DKG $+ N \times$ registration $+ N \times$ ballot create $+ N \times$ ballot verify $+$ tally). Real deployments will parallelise both ballot construction (one voter per voter device) and verification (a verifier pool), but the serialized figure is the worst-case ceiling and the right number to anchor sizing decisions.

Table 6.1: Headline performance numbers for three target deployment scales (AWS run, single-threaded). All per-phase numbers come from Table 6.8; the rightmost column is the serialized end-to-end total. Tally is the full pipeline including BSGS lookup.

Scenario	DKG	Voter reg.	Ballot create	Ballot verify	Tally	Serial e2e
Boardroom ($N = 256$)	1.06 ms	116 µs	13.2 ms	4.27 ms	200 ms	≈ 4.7 s
Small org ($N = 4{,}096$)	1.06 ms	118 µs	157 ms	37.4 ms	4.33 s	≈ 13.3 min
Mid-scale ($N = 16{,}384$)	1.09 ms	118 µs	668 ms	145 ms	31.8 s	≈ 3.7 h

The serialized end-to-end figure is dominated by per-voter ballot construction at every scale — $\approx 72\%$ of the boardroom total, $\approx 80\%$ at small-org scale, and $\approx 82\%$ at mid-scale — which is the cost paid in parallel by the voter pool, not by any centralised infrastructure. The non-parallelisable components (DKG once, full-pipeline tally once) stay under 32 seconds even at the largest measured scale.

6.3 Per-Phase Microbenchmarks

This section reports the cost of each individual cryptographic operation in isolation, before they are composed into ballots or full elections.

6.3.1 Key Generation Costs

The distributed key generation (Phase 1 of Chapter 3) involves all η authorities running Pedersen-Feldman VSS interactively. For the configuration tested ($\eta = 3$, $t = 2$), DKG is essentially constant in election size N — it depends only on the number of authorities and the threshold, not on the voter count. Measured wall-clock times across the AWS run are: 1.130 ms at $N = 16$, 1.065 ms at $N = 64$, 1.060 ms at $N = 256$, 1.062 ms at $N = 1{,}024$, 1.060 ms at $N = 4{,}096$, and 1.091 ms at $N = 16{,}384$ (mean $\pm 0.5\%$ across runs). The variation across N is well within measurement noise; DKG is genuinely $O(\eta^2 \cdot t)$ in the authority parameters, independent of voters. Figure 6.1 confirms the constancy visually: the dashed reference line is the mean over all six measurements, and no individual measurement deviates by more than $\sim 7\%$.

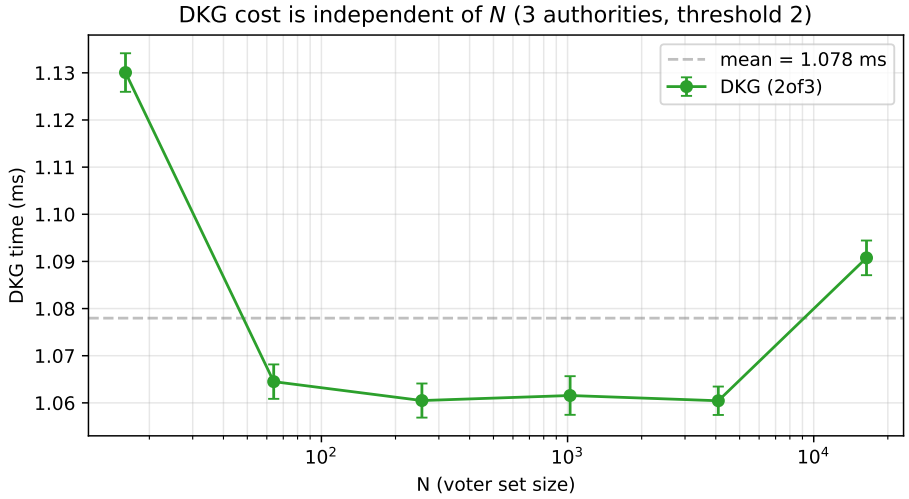

Figure 6.1: Distributed-key-generation cost is independent of voter set size N (AWS run, $\eta = 3$ authorities, threshold $t = 2$). Error bars are Criterion 95% confidence intervals.
Data: `thesis/results/criterion/N{X}_dkg/2of3/new/estimates.json`.

This is consistent with the protocol design: each authority performs $O(t)$ scalar multiplications for its own polynomial commitments, $O(\eta)$ share evaluations, and $O(\eta \cdot t)$ verifications of received

shares. For the small-cohort case ($\eta \leq 10$) typical of real elections, DKG is a one-time cost in the low milliseconds.

6.3.2 Schnorr-Type Proof Costs

The four Schnorr-style proving systems (representation, DLEQ, encryption-validity, serial-validity) take constant time per proof, with no dependency on election parameters. Their measured prove and verify times are summarized in Table 6.2.

Table 6.2: Per-proof prove and verify times for the four Schnorr-style proving systems. Times are mean ± half-confidence-interval percentage (Criterion).

Proof system	Prove	Verify
Representation ($n = 1$ generator)	43.70 µs ±0.5%	74.73 µs ±0.5%
Representation ($n = 2$ generators)	57.53 µs ±0.4%	83.37 µs ±0.5%
Representation ($n = 4$ generators)	86.85 µs ±0.5%	101.7 µs ±0.4%
Representation ($n = 8$ generators)	139.4 µs ±0.4%	138.0 µs ±0.5%
DLEQ	85.71 µs ±0.4%	146.3 µs ±0.5%
Encryption validity	120.4 µs ±0.5%	177.9 µs ±0.4%
Serial validity	191.5 µs ±0.4%	280.6 µs ±0.4%

The four times scale with the number of group operations in the underlying relation: representation is the simplest (two generators, two scalars), DLEQ adds a second relation, encryption-validity proves an ElGamal encryption (three generators), and serial-validity is the most complex (four group elements, three witnesses, two relations). All four operations sit comfortably in the sub-millisecond range; even on commodity hardware, a voter constructing a ballot with these proofs adds well under a millisecond of total Schnorr work.

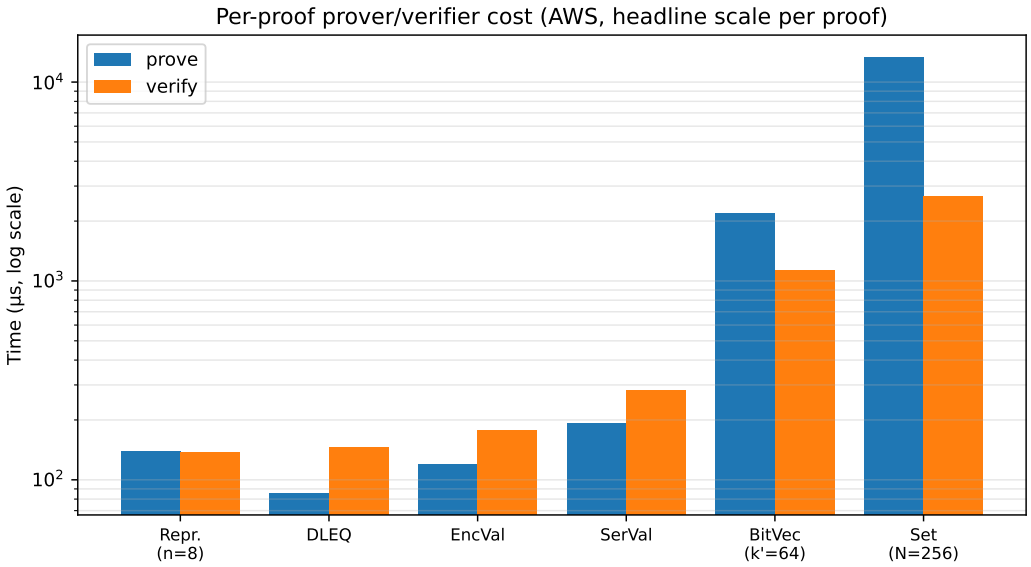

Figure 6.2: Per-proof prover vs. verifier cost at the largest scale measured for each proof family (AWS run), plotted on a log y-axis. Representation ($n = 8$), DLEQ, encryption-validity and serial-validity all stay in the low hundreds of µs; bit-vector ($k' = 64$) and commitment-set ($N = 256$) are an order of magnitude heavier. Data: `thesis/results/timings.csv (source=aws)`.

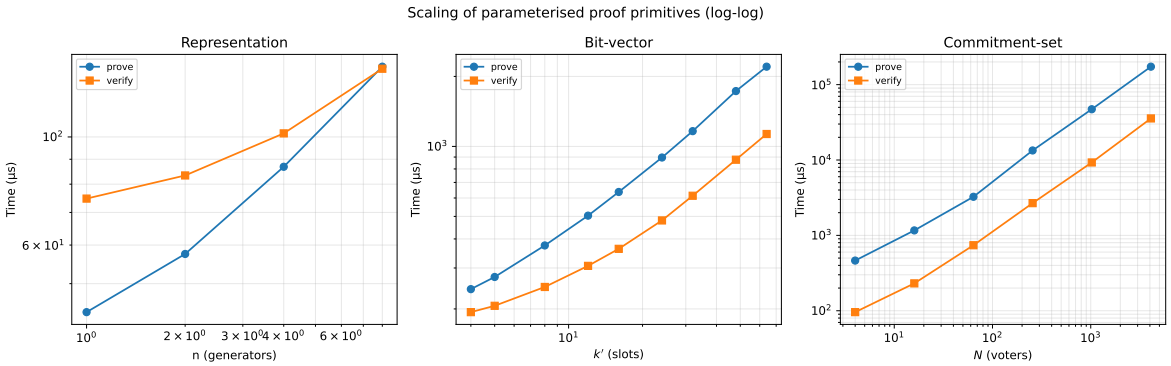

*Figure 6.3: Log–log scaling of the three parameterised proof primitives. **Left:** representation proof prove/verify times across $n \in \{1, 2, 4, 8\}$ generators. **Middle:** bit-vector proof across padded slot counts $k' \in \{4, 5, 8, 12, 16, 24, 32, 48, 64\}$. **Right:** commitment-set proof across $N \in \{4, 16, 64, 256, 1{,}024, 4{,}096\}$. All three families show clean linear or near-linear scaling on log axes. Data: `thesis/results/timings.csv (source=aws)`.*

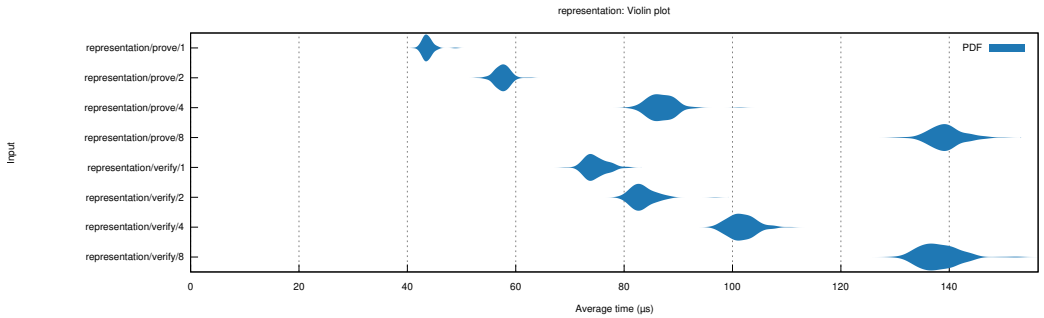

Figure 6.4: Criterion violin plot for the representation proof, showing the distribution of measured samples at each generator count n (`prove`). The tight density at every n confirms low measurement noise on the AWS instance.
Data: `thesis/results/criterion/representation/report/violin.svg`.

6.3.3 Bit-Vector Proof Costs

The bit-vector proof (Section 3.7, statement P3) asserts that an encrypted bit vector contains only zeros and ones, with a fixed weight. Its cost scales linearly with the padded slot count k'.

Table 6.3: Bit-vector proof prove and verify times as a function of padded slot count k'.

k'	Prove	Verify
4	243.3 µs ±0.4%	193.7 µs ±0.6%
5	274.5 µs ±0.5%	206.3 µs ±0.5%
8	375.0 µs ±0.5%	248.6 µs ±0.5%
12	503.8 µs ±0.4%	306.4 µs ±0.5%
16	636.8 µs ±0.5%	362.6 µs ±0.5%
24	895.9 µs ±0.4%	480.4 µs ±0.4%
32	1.163 ms ±0.4%	613.5 µs ±0.5%
48	1.730 ms ±2.3%	876.6 µs ±0.4%
64	2.203 ms ±0.5%	1.131 ms ±0.4%

The doubling pattern is visible: from $k' = 4$ to $k' = 64$ (a $16\times$ increase), prove time grows from 243 µs to 2.20 ms ($9.1\times$), and verify time grows from 194 µs to 1.13 ms ($5.8\times$). The sub-linear growth on the verifier side reflects the multi-scalar multiplication used at the verification step, which has more aggressive amortization than a naive sum.

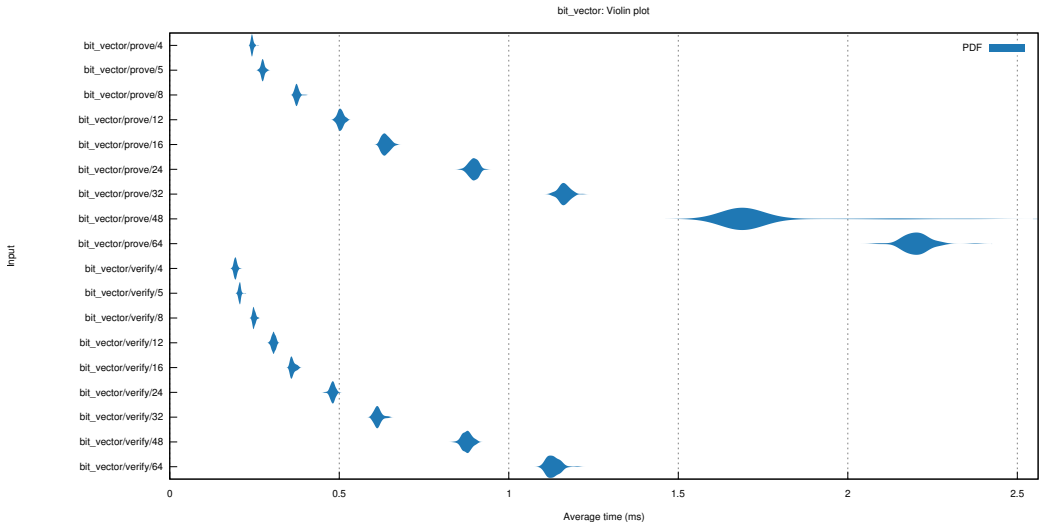

Figure 6.5: Criterion violin plot for the bit-vector proof across padded slot counts $k' \in \{4, 5, 8, 12, 16, 24, 32, 48, 64\}$ (`prove`). The roughly linear envelope is consistent with the $O(k')$ scaling reported in Table 6.3.
Data: `thesis/results/criterion/bit_vector/report/violin.svg`.

6.3.4 Commitment-Set (Eligibility) Proof Costs

The commitment-set proof (Section 3.7, statement P1) is the largest individual component of an Aura ballot. It is also the most parameter-sensitive: its cost depends on both the voter list size N and the n-ary digit base used in the Bootle-Spark construction.

Table 6.4: Commitment-set proof prove and verify times for binary ($n = 2$) versus quaternary ($n = 4$) digit base, across the full range of N at which the standalone bench was run. The quaternary variant approximately halves prove time at essentially the same verify time. The commitment-set proof at $N = 16{,}384$ is captured indirectly via the ballot-creation cost in Table 6.8 (where it is the dominant component); no standalone $N = 16{,}384$ commitment-set bench was run.

N	Variant	Prove	Verify
4	$n = 2$	463.2 µs ±0.3%	95.84 µs ±0.3%
16	$n = 2$	1.164 ms ±0.6%	230.8 µs ±0.3%
64	$n = 2$	3.251 ms ±0.3%	740.0 µs ±0.3%
64	$n = 4$	2.115 ms ±0.4%	694.2 µs ±0.3%
256	$n = 2$	13.39 ms ±0.4%	2.677 ms ±0.3%
256	$n = 4$	7.123 ms ±0.3%	2.541 ms ±0.2%
1,024	$n = 2$	47.16 ms ±0.3%	9.276 ms ±0.2%
1,024	$n = 4$	22.94 ms ±0.2%	8.886 ms ±1.2%
4,096	$n = 2$	173.1 ms ±0.2%	35.60 ms ±0.2%

Two observations dominate this table.

First, the prove and verify times grow cleanly with N, but at very different rates. Going from $N = 4$ to $N = 4{,}096$ (a $1024\times$ increase in voter count), prove time grows from 0.46 ms to 173 ms (a

$\approx 374\times$ increase), and verify time grows from 0.10 ms to 35.6 ms ($\approx 372\times$). Both are slower than the asymptotic claim of $O(\log N)$ in proof size — which is only about the byte size of the proof, not the cost of producing it. The actual prover and verifier work in the Bootle construction grows as $O(N)$ in the worst case, since both parties must touch every commitment in the list at least once. What the GK construction reduces is the *communication*, not the overall computation. This is consistent with what is reported in the literature.

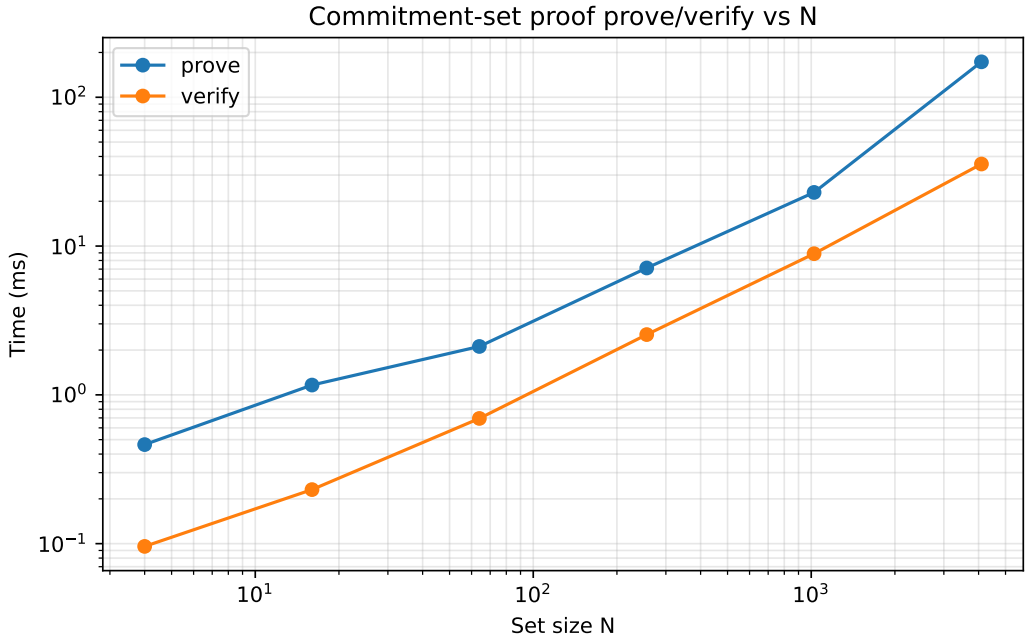

Figure 6.6: Commitment-set proof prove and verify times as a function of voter set size N, on a log-log scale (AWS run, binary digit base $n = 2$). The near-linear lines indicate close-to-$O(N)$ growth, with verification cheaper than proving across the whole range.
Data: `thesis/results/timings.csv (group=commitment_set)`.

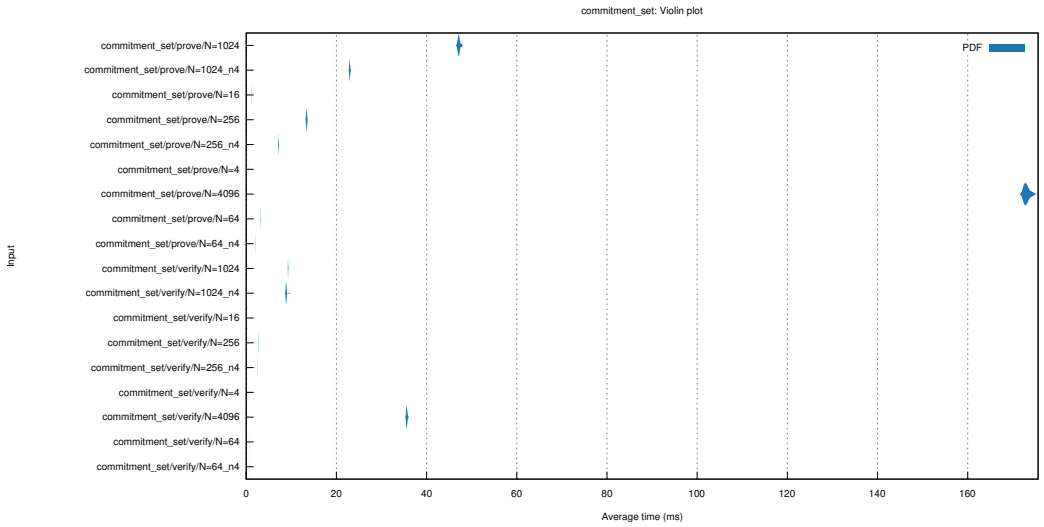

Figure 6.7: Criterion violin plot for the commitment-set proof prover across all measured N on the AWS run. The per-sample dispersion is tight; the medians span more than two orders of magnitude as N grows from 4 to $4{,}096$.
Data: `thesis/results/criterion/commitment_set/report/violin.svg`.

Second, the choice of n-ary digit base meaningfully affects prover cost. Compared at the same N, the $n = 4$ variant approximately halves prove time: 7.12 ms versus 13.39 ms at $N = 256$, 22.94 ms versus 47.16 ms at $N = 1{,}024$. Verify times are essentially identical. The intuition is that with $n = 4$ the prover commits to $\log_4 N$ quaternary digits instead of $\log_2 N$ binary digits, halving the commitment-and-bit-proof rounds even though each round becomes slightly more expensive. Verification work is dominated by the same multi-scalar multiplication regardless of base, so it is largely unchanged.

6.3.5 Verification Costs

A full ballot verification combines the per-proof verifications above. The dominant component is the commitment-set proof verification; the encryption-validity, serial-validity, and bit-vector verifications add a few hundred microseconds each. Full ballot verification times are reported in Section 6.4 alongside the rest of the election lifecycle.

A separate question is whether *batch* verification of multiple ballots improves over verifying them individually. The Aura paper claims that batch verification of compatible Schnorr-type proofs amortizes per-proof cost into a single multi-scalar multiplication. Our measurements (Table 6.5) confirm this for all three batchable proof systems (representation, DLEQ, encryption-validity).

Table 6.5: Batch-verification amortization. "Individual" is the time to verify each proof one at a time; "Batched" is the time for a single combined multi-scalar multiplication over all proofs at once. Speedup is the ratio.

Proof system	Batch size	Individual	Batched	Speedup
Representation	1	80.83 µs	58.06 µs	1.39×
Representation	10	797.2 µs	430.7 µs	1.85×
Representation	25	2.027 ms	1.044 ms	1.94×
Representation	50	4.041 ms	2.012 ms	2.01×
Representation	100	8.166 ms	3.700 ms	2.21×
DLEQ	1	145.2 µs	80.54 µs	1.80×
DLEQ	10	1.442 ms	631.2 µs	2.28×
DLEQ	25	3.585 ms	1.550 ms	2.31×
DLEQ	50	7.188 ms	2.812 ms	2.56×
DLEQ	100	14.38 ms	5.249 ms	2.74×
Enc-validity	1	178.8 µs	94.22 µs	1.90×
Enc-validity	10	1.772 ms	757.3 µs	2.34×
Enc-validity	25	4.446 ms	1.840 ms	2.42×
Enc-validity	50	8.936 ms	3.268 ms	2.73×
Enc-validity	100	17.92 ms	6.029 ms	2.97×

The speedup grows with batch size, and reaches 2–3× at batch size 100 across all three proof systems. The intuition is that random linear combination compresses many independent verification equations into a single multi-scalar multiplication, after which Pippenger amortization makes the marginal cost of each additional proof very small. For a real election with thousands of ballots, batch verification is the difference between minutes and tens of seconds of total verifier work.

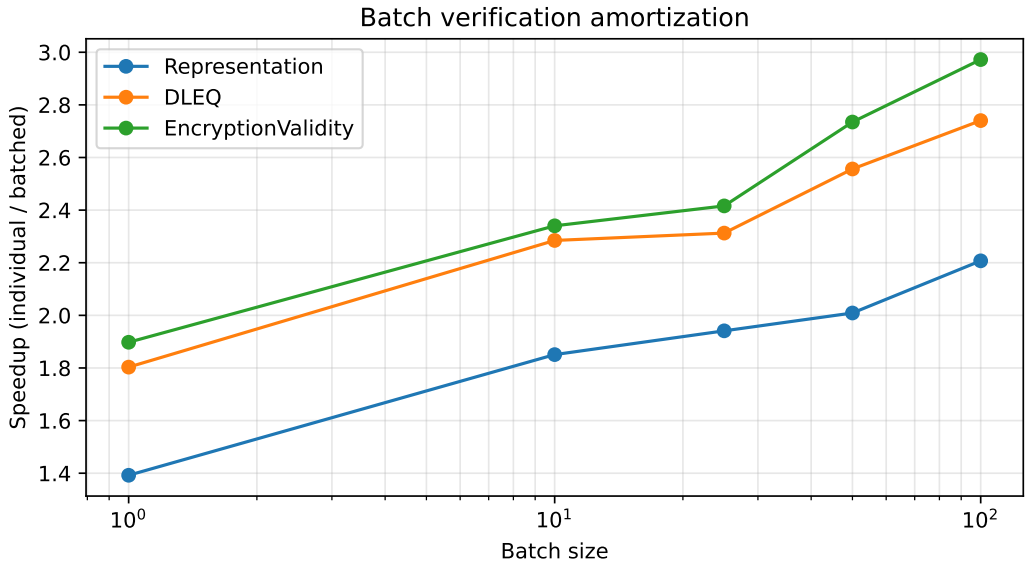

Figure 6.8: Batch-verification speedup (individual ÷ batched) as a function of batch size on the AWS run. All three batchable proof systems show monotonically increasing speedup, plateauing in the 2–3× range at batch size 100.
Data: `thesis/results/timings.csv (group=batch_verify_*)`.

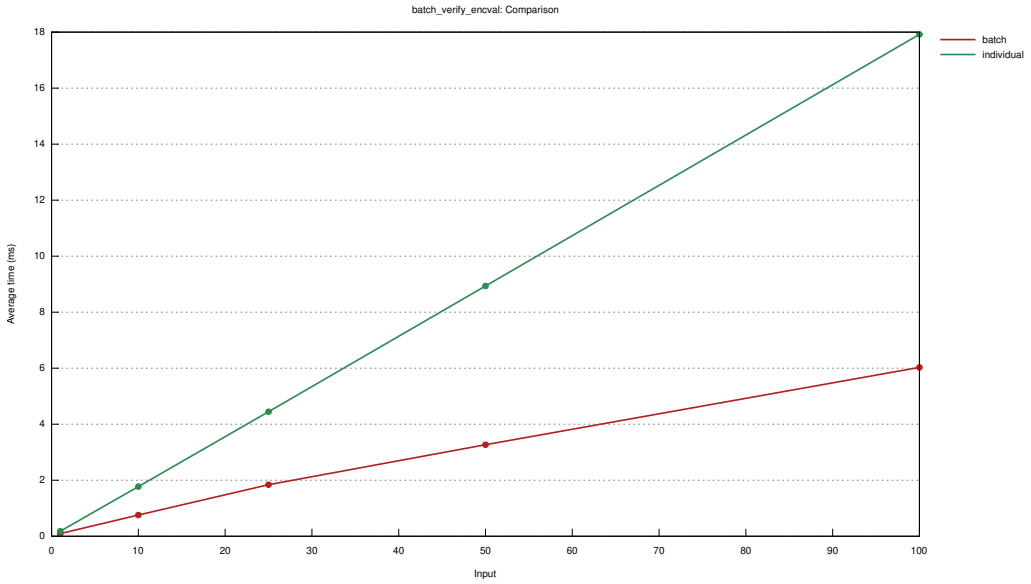

Figure 6.9: Criterion's per-batch "lines" plot for the encryption-validity batch verifier, showing measured time as a function of batch size with `individual` (orange) and `batch` (blue) lines on the same axes.
Data: `thesis/results/criterion/batch_verify_encval/report/lines.svg`.

Batch verification at election scale. The amortization above is measured on standalone proofs with no election context. A more deployment-relevant question is: how long does it take to batch-verify k complete ballots, each one referring to an N-voter commitment list? Table 6.6 reports that measurement for $N \in \{16, 64, 256, 1{,}024, 4{,}096, 16{,}384\}$ and batch sizes $k \in \{1, 10, 50, 100\}$. The per-ballot cost in this table is dominated by the commitment-set verifier inside each ballot, so the total time grows roughly linearly in both N and k — batch verification compresses the *constant-overhead*

portion of each verification, but the $O(N)$ commitment-set work cannot be folded into a single multi-scalar multiplication.

Table 6.6: Election-scale full-ballot batch verification (AWS run): wall-clock time to verify k complete ballots at once for a voter list of size N. Each row is one election scale; each column is one batch size; empty cells mark configurations that were not executed.

N	$k = 1$	$k = 10$	$k = 50$	$k = 100$
16	1.623 ms	16.43 ms	—	—
64	2.155 ms	21.77 ms	109.5 ms	—
256	4.321 ms	43.28 ms	216.1 ms	439.4 ms
1,024	10.62 ms	107.3 ms	537.9 ms	1.073 s
4,096	37.14 ms	374.4 ms	1.861 s	3.732 s
16,384	145.4 ms	1.456 s	7.301 s	14.58 s

Two practical points fall out of this table. First, the *column-wise* ratio is close to constant: doubling N roughly doubles per-row time across every batch size, confirming that the commitment-set verifier inside each ballot is what scales with N. Second, the *row-wise* per-ballot cost (each cell divided by its k) drops modestly as k grows — at $N = 16{,}384$, single-ballot verification is 145 ms but per-ballot cost at $k = 100$ is ≈ 146 ms, essentially unchanged. The Schnorr-style batch amortization is real but masked at large N because the commitment-set $O(N)$ work cannot be batched, only the constant per-ballot Schnorr work can. This is the central practical reason a 10^4-voter deployment will deploy a parallel verifier pool rather than relying on Schnorr batching alone.

6.3.6 Tally Costs

The tally pipeline has two phases (Section 3.10): serial-number decryption (used for deduplication) followed by aggregate decryption (used to recover the final tally). Both involve the same threshold-decryption primitive, applied to different ciphertexts.

Table 6.7: Tally pipeline timings as a function of voter count N. "Serial decrypt per tallier" is the wall-clock time for one tallier to perform her threshold-decryption share for all N ballot serial numbers. "Full pipeline" is the end-to-end tally including aggregate decryption and BSGS lookup.

N	Serial decrypt per tallier	Full pipeline
16	1.832 ms ±0.3%	14.57 ms ±0.3%
64	7.373 ms ±0.5%	50.96 ms ±0.5%
256	29.36 ms ±0.4%	200.3 ms ±0.2%
1,024	118.2 ms ±0.4%	888.0 ms ±0.5%
4,096	474.4 ms ±0.4%	4.334 s ±0.3%
16,384	2.004 s ±0.3%	31.78 s ±0.2%

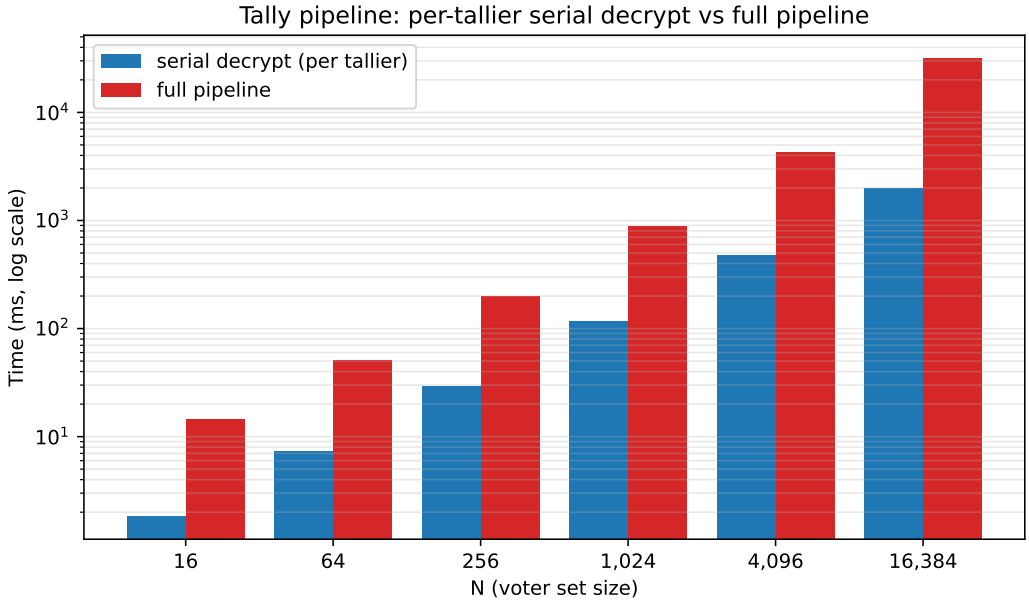

Figure 6.10: Tally breakdown across N: per-tallier serial-decrypt cost (blue) and full-pipeline cost (red) on the AWS run, log y-axis. The full pipeline stays roughly an order of magnitude above the serial step throughout, indicating aggregate decryption + BSGS lookup dominate over the per-ballot partial decryption phase. Data: `thesis/results/timings.csv (group ends with _tally)`.

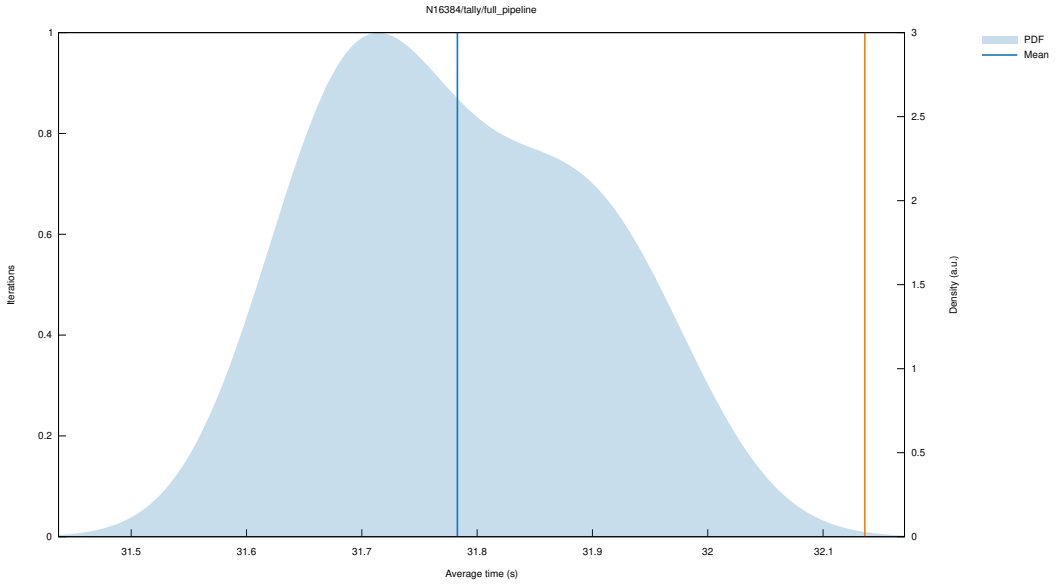

Figure 6.11: Criterion sample-density plot for the full tally pipeline at $N = 16{,}384$. The narrow peak around 31.8 s confirms the reported $\pm 0.2\%$ confidence in Table 6.7. Data: `thesis/results/criterion/N16384_tally/full_pipeline/report/pdf.svg`.

Serial decryption scales linearly with N: each ballot has its own encrypted serial number, and the tallier must partial-decrypt all of them. From $N = 16$ to $N = 16{,}384$ (a $1024\times$ increase), per-tallier serial decryption grows from 1.83 ms to 2.00 s ($\approx 1094\times$), close to the linear expectation; the slight super-linearity at the top of the range is explained by cache pressure from the larger ciphertext list on the 2 MiB-per-vCPU AWS instance.

The full pipeline is roughly $8\times$ slower than per-tallier serial decryption alone across the measured range, reflecting the additional work of aggregate decryption (per-candidate ciphertexts), Lagrange combination of tallier shares, and the BSGS lookup to convert g^{T_j} back into the integer T_j. At $N = 256$ the full pipeline takes 200 ms; at $N = 4{,}096$ it is 4.3 s; at $N = 16{,}384$ it is 31.8 s. Linear extrapolation from the measured $N = 16{,}384$ point puts a single-instance tally at $N = 10^6$ near ≈ 32 minutes; we mark this an extrapolation rather than a measurement because the $N = 65{,}536$ attempt ran out of memory on this hardware and could not be timed.

6.4 End-to-End Election Simulation

This section combines the per-phase costs into end-to-end timing for a complete election across the measured voter scales. The configuration is $\eta = 3$ authorities with threshold $t = 2$, default election parameters, and $N \in \{16, 64, 256, 1{,}024, 4{,}096, 16{,}384\}$.

Table 6.8: End-to-end election lifecycle timings (AWS run). "Voter reg." is the per-voter registration cost (so total registration is roughly $N\times$ that value). "Ballot create" and "Ballot verify" are single-ballot times; total election ballot work is roughly $N\times$ those values.

N	DKG	Voter reg.	Ballot create	Ballot verify	Tally
16	1.130 ms	119.2 µs	3.011 ms	1.622 ms	14.57 ms
64	1.065 ms	117.0 µs	5.095 ms	2.160 ms	50.96 ms
256	1.060 ms	116.2 µs	13.20 ms	4.266 ms	200.3 ms
1,024	1.062 ms	117.6 µs	42.43 ms	10.64 ms	888.0 ms
4,096	1.060 ms	118.0 µs	156.7 ms	37.35 ms	4.334 s
16,384	1.091 ms	118.4 µs	668.2 ms	145.2 ms	31.78 s

Reading across rows, several patterns emerge:

- **DKG is constant in** N, as expected (it depends only on η and t): the measured value sits between 1.06 ms and 1.13 ms across all six scales.
- **Voter registration is constant in** N, because each voter's work (one Pedersen commitment, one representation proof) is independent of how many other voters there are; per-voter cost stays in a tight 116–119 µs band across the full range.
- **Ballot creation grows with** N, dominated by the commitment-set proof. From $N = 16$ to $N = 16{,}384$ (a $1024\times$ increase), ballot creation grows from 3.01 ms to 668 ms ($\approx 222\times$) — much slower than N itself but faster than $\log N$ alone, consistent with the $O(N/\log N)$ amortized cost of Pippenger multi-scalar multiplication in the prover.
- **Ballot verification grows even more slowly**: from 1.62 ms at $N = 16$ to 145 ms at $N = 16{,}384$ ($\approx 90\times$ over the same $1024\times$ scale). The verifier's MSM amortizes more aggressively, and the bit-vector and Schnorr verifications are constant in N.

- **Tally grows linearly** (Section 6.3.6).

The aggregate picture: a boardroom-scale election with $N = 256$ takes ≈ 1.1 ms for setup, 30 ms total voter registration (256×116 µs), 3.4 s total ballot construction (256×13.20 ms; parallelizable across voters), 1.1 s total ballot verification (256×4.27 ms; much less with batch verification, see Table 6.5), and 0.20 s for tally. Total serialized wall-clock time is under 5 seconds; in practice ballots are constructed in parallel by different voters and the protocol is dominated by verifier work, which on a single AWS vCPU sits near a second.

Extending the same arithmetic to $N = 4{,}096$ gives ≈ 1.1 ms DKG, 0.48 s total registration, 642 s (≈ 10.7 min) total serialized ballot construction, 153 s (≈ 2.6 min) total serialized ballot verification, and 4.3 s tally; batch verification compresses the verifier total significantly (Section 6.3.5). At $N = 16{,}384$ those become 3.04 h serialized ballot construction ($16{,}384 \times 668.2$ ms), 39.6 min ballot verification ($16{,}384 \times 145.2$ ms), and 31.8 s tally — numbers that argue for a parallel ballot pool and for batched verification in any deployment beyond the boardroom scale. Note that at this N the Schnorr-style batch verification of Section 6.3.5 no longer compresses the verifier total appreciably (Table 6.6), because the commitment-set verifier inside each ballot is $O(N)$ and cannot be folded into the batched multi-scalar multiplication.

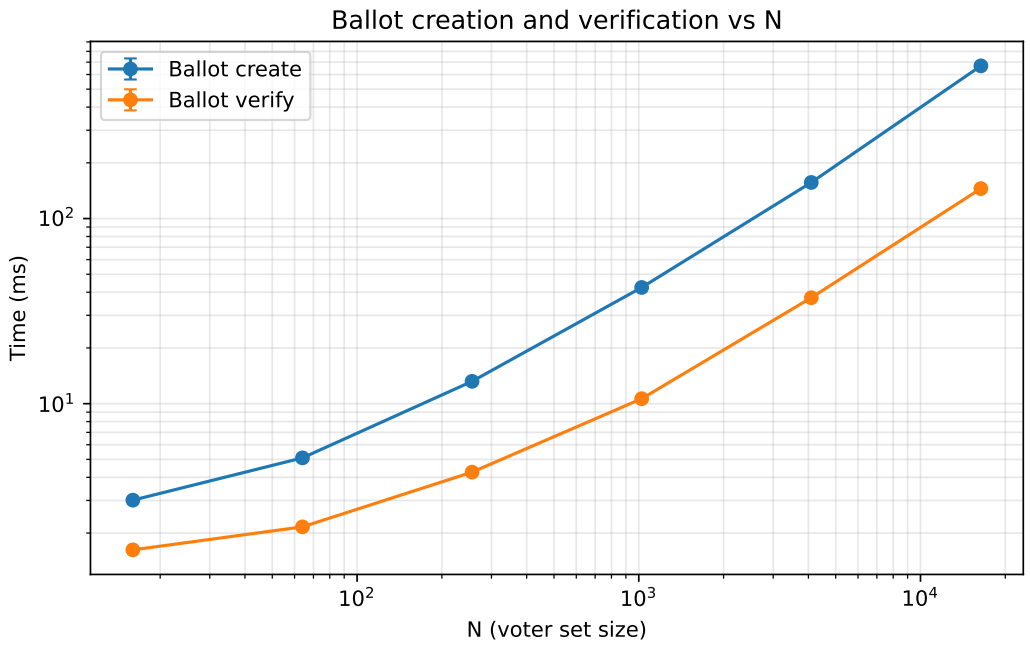

Figure 6.12: Per-ballot create and verify times as a function of N on a log-log scale (AWS run). Both grow super-linearly in raw N but sub-linearly in $\log_2 N$, consistent with the $O(N/\log N)$ Pippenger amortization argued in the text.
Data: `thesis/results/timings.csv (group=N*_ballot)`.

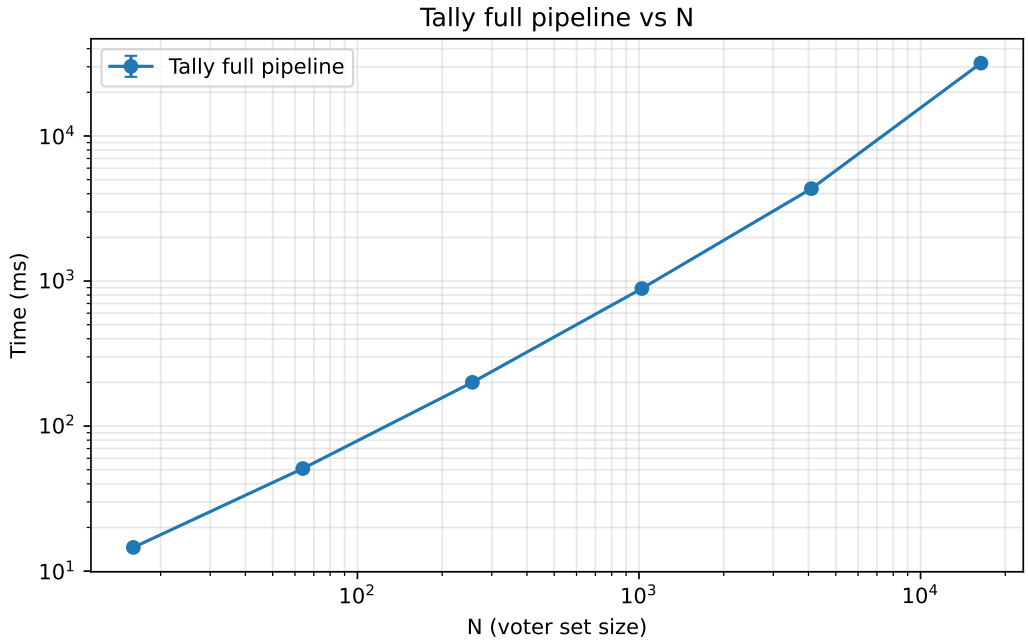

Figure 6.13: Tally full-pipeline cost as a function of N on log-log axes (AWS run). The straight slope confirms the linear-in-N prediction across more than three decades of voter set size.
Data: `thesis/results/timings.csv (group=N*_tally, function=full_pipeline)`.

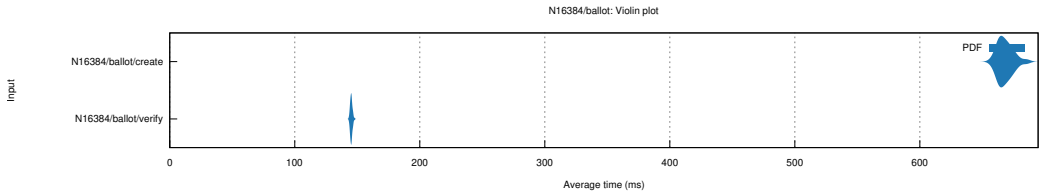

Figure 6.14: Criterion violin plot for the $N = 16{,}384$ ballot group on the AWS run, showing the sample distributions for `create` and `verify`. The fully-resolved distributions visualise the $\pm 0.3\%$ / $\pm 0.2\%$ confidence figures reported in Table 6.8.
Data: `thesis/results/criterion/N16384_ballot/report/violin.svg`.

6.5 Proof Size Analysis

Proof and ballot sizes are independent of runtime cost and were measured across the entire supported range up to $N = 1{,}048{,}576$ voters.

6.5.1 Individual Proof Sizes

The four Schnorr-style proofs have constant size, independent of any election parameter:

Table 6.9: Sizes of the four Schnorr-style proofs.

Proof system	Size
Representation	96 B
DLEQ	96 B
Encryption validity	128 B
Serial validity	192 B

The bit-vector proof grows linearly with k' (since each padded slot contributes its own committed bit):

Table 6.10: Bit-vector proof size as a function of padded slot count k'.

k'	Size	k'	Size
4	256 B	16	640 B
5	288 B	32	1,152 B
8	384 B	64	2,176 B

Even at $k' = 64$ slots, the bit-vector proof is just over 2 KB — far smaller than running 64 separate bit proofs would be.

6.5.2 Commitment-Set Proof Size

This is the most interesting size measurement: the Bootle-Spark construction promises $O(\log N)$ proof size, and our measurements confirm this scaling clearly.

Table 6.11: Commitment-set proof size as a function of voter count N, using binary ($n = 2$) digit decomposition. The size grows by exactly 320 bytes for each doubling of N, confirming $O(\log N)$ scaling.

N	(n, m)	Size	N	(n, m)	Size
16	$(2, 4)$	672 B	16,384	$(2, 14)$	2,272 B
64	$(2, 6)$	992 B	65,536	$(2, 16)$	2,592 B
256	$(2, 8)$	1,312 B	262,144	$(2, 18)$	2,912 B
1,024	$(2, 10)$	1,632 B	1,048,576	$(2, 20)$	3,232 B
4,096	$(2, 12)$	1,952 B			

The pattern is clear and exact: each doubling of N (one additional binary digit) adds exactly 320 bytes to the proof. Going from $N = 16$ to $N = 10^6$ — a $65{,}000\times$ increase in voter count — proof size grows only $4.81\times$, from 672 B to 3.23 KB. This is the core efficiency property of the GK/Bootle construction in practice.

6.5.3 Full Ballot Sizes

A full Aura ballot includes the encryption validity proofs (one per slot), the bit-vector proof, the encrypted serial number with its validity proof, and the commitment-set proof. The total scales predominantly with N (via the commitment-set proof), with smaller contributions from k'.

Table 6.12: Total ballot size as a function of voter count N, with default election parameters ($k = 4$, $\ell_{\min} = 1$, $\ell_{\max} = 2$, giving $k' = 5$). Each doubling of N adds 320 bytes, the same as the commitment-set proof, confirming that proof is the dominant N-dependent term.

N	Ballot size	N	Ballot size
16	2,272 B	16,384	3,872 B
64	2,592 B	65,536	4,192 B
256	2,912 B	262,144	4,512 B
1,024	3,232 B	1,048,576	4,832 B
4,096	3,552 B		

A million-voter election produces ballots under 5 KB each, despite the $N = 10^6$ commitment list. For comparison, a typical TLS handshake is several KB; an Aura ballot is the same order of magnitude as a single web-page request.

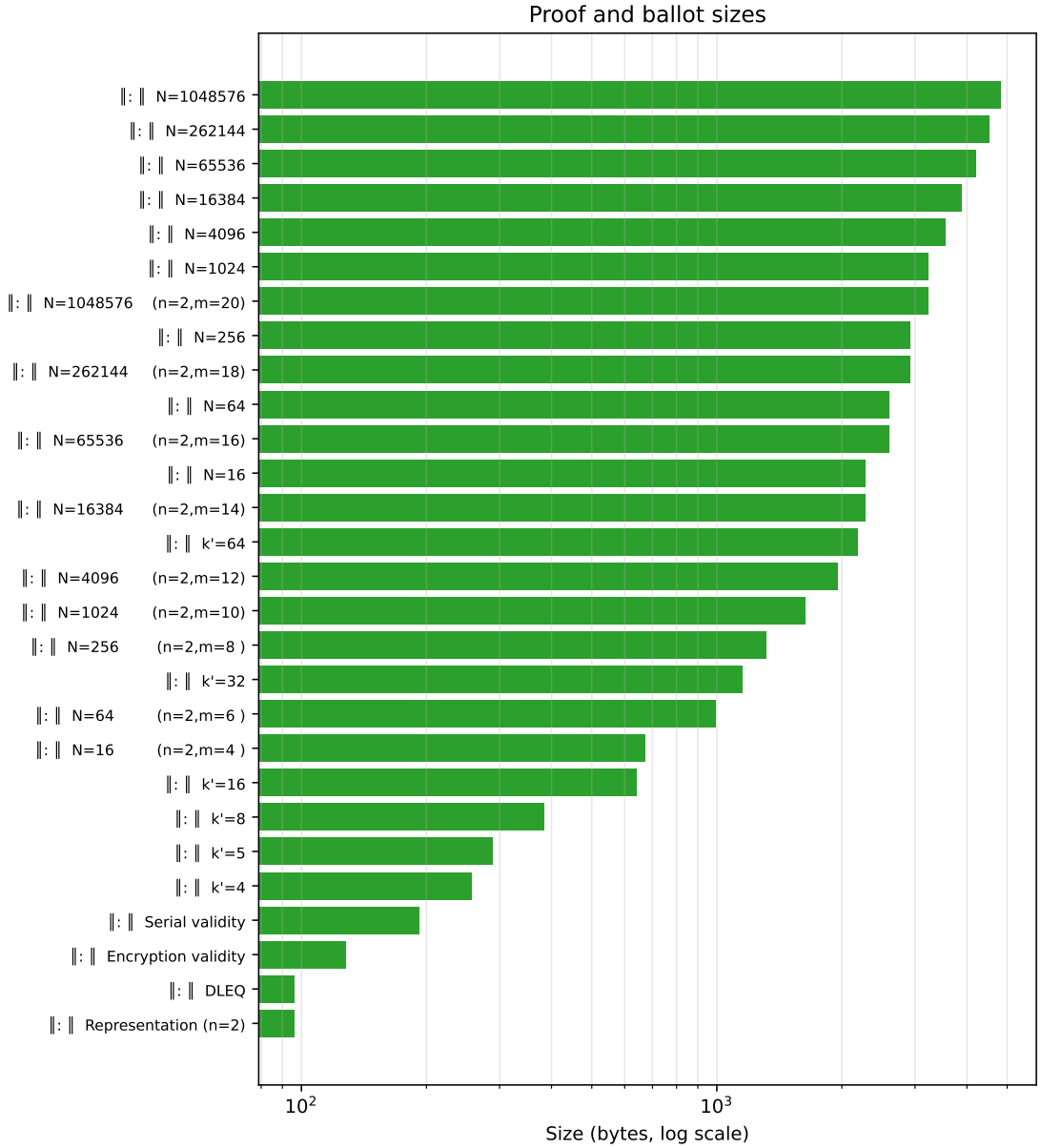

Figure 6.15: Proof and ballot byte sizes from `proof_sizes.rs` *(log x-axis). The four Schnorr-style proofs and bit-vector proofs sit at the low end (96–2,176 B); commitment-set proofs grow with* $\log N$*; full ballot sizes scale from* 2.2 *KB at* $N = 16$ *to under* 5 *KB at* $N = 10^6$*.*
Data: `thesis/results/sizes.csv (from raw/proof_sizes.log)`*.*

6.6 Comparison with the Original Paper's Estimates

The original Aura paper (Section 4.1) gives only asymptotic complexity, not concrete numbers. We can therefore not directly compare wall-clock timings, but we can verify that the asymptotic claims hold in practice.

Claim 1: Commitment-set proof size is $O(\log N)$. The paper writes: *"The size of this proof scales as $O(\log(N_{voters}))$ using the instantiation referenced."* Our measurements (Table 6.11) confirm this exactly: every doubling of N adds a constant 320 bytes to the proof, which is $\Theta(\log N)$ growth.

Going from $N = 16$ to $N = 10^6$, the proof grows $4.81\times$ while N grows $65{,}000\times$ — a near-perfect logarithmic relationship.

Claim 2: Verification scales as $O(N/\log N)$ via Pippenger. The paper writes: *"While verification at first appears to scale as $O(N_{voters})$, the use of efficient multiscalar multiplication algorithms can reduce this complexity to $O(N_{voters}/\log(N_{voters}))$ with good constants."* Our measurements show commitment-set verification growing from 230 µs at $N = 16$ to 35.60 ms at $N = 4{,}096$ (a $\approx 154\times$ increase for a $256\times$ increase in N). Naive $O(N)$ scaling would predict a $256\times$ slowdown; the asymptotic $O(N/\log_2 N)$ amortisation that Pippenger MSM theoretically achieves predicts a $(4096/\log_2 4096)/(16/\log_2 16) = (4096/12)/(16/4) \approx 85\times$ slowdown. The measured $\approx 154\times$ sits cleanly between these two bounds, much closer to the asymptotic ideal than to naive linear scaling — the gap from $85\times$ to $154\times$ reflects the constant factors and the non-MSM work (transcript management, commitment computations, serialization) that Pippenger amortisation cannot fold away.

Claim 3: Batch verification has constant marginal cost. The paper writes: *"Verifier weighting of common group elements in the required multiscalar multiplication evaluation makes the marginal verification complexity constant, amortizing the overall cost across the batch."* Our measurements (Table 6.5) confirm this clearly: at batch size 100, marginal per-proof verification cost drops by 2–3× across all three batchable proof systems. For encryption-validity specifically, batched cost is 60.29 µs/proof at batch 100 (Table 6.5), versus 179.2 µs/proof individually — exactly the amortization the paper predicts.

An empirical contribution: the n-ary trade-off. The Aura paper does not benchmark different choices of n in the $n^m = N$ parameterization. Our measurements (Table 6.4) provide concrete data: the $n = 4$ variant approximately halves prover time while leaving verifier time unchanged. This is a meaningful practical observation, since prover time is the per-voter cost (paid once per ballot, on possibly resource-constrained client devices) while verifier time is the per-ballot infrastructure cost (paid centrally, amortizable). Choosing $n = 4$ over $n = 2$ is a Pareto improvement for client-side performance at no infrastructure cost.

Summary. All three of the paper's quantitative claims hold in our implementation. The asymptotic predictions are not just theoretical guarantees but practical reality: the implementation behaves exactly as the paper describes, with the additional empirical finding that the $n = 4$ digit base choice is strictly better than $n = 2$ on the tested platform.

7. DISCUSSION

This chapter steps back from the implementation details of Chapter 5 and the measurements of Chapter 6 to consider what the work means in practice. We first ask whether Aura, as implemented, is viable for the kinds of elections the introduction motivated. We then catalog what Aura deliberately does not solve, what limitations remain in the present implementation, and which directions seem most promising for follow-up work.

7.1 Practical Viability for Direct Democracy

The benchmarks of Chapter 6 let us answer the practical question concretely: at what election scale does Aura become workable, and what does each role pay?

Voter-side cost. The voter's wall-clock cost is dominated by the commitment-set (eligibility) proof, which scales with the voter list size N. Measured on the AWS instance (2 vCPU Xeon Platinum 8488C): at $N = 256$ a ballot takes 13.2 ms to construct, at $N = 1{,}024$ 42.4 ms, at $N = 4{,}096$ 157 ms, and at $N = 16{,}384$ 668 ms. Extrapolating linearly in N from the measured $N = 16{,}384$ point places $N = 10^6$ ballot construction near ≈ 41 s on the same single-vCPU hardware; we mark this an extrapolation rather than a measurement because the $N = 65{,}536$ attempt exhausted the 3.7 GiB instance memory and could not be timed. For a phone-grade CPU we would expect proof times perhaps 5–10$\times$ longer (an estimate rather than a measurement; no ARM benchmarks were run for this thesis). This places Aura comfortably in the practical range for elections up to mid-six-figure voter counts (university elections, professional-society votes, municipal referenda) on commodity client devices, and within reach for city- to country-scale elections if voters are willing to wait a minute or two for a ballot to be prepared — comparable to the time spent in a physical polling booth.

The empirical observation that $n = 4$ approximately halves prover time at no verifier cost (Section 6.3.4) is particularly valuable here, since prover work is precisely the client-side cost paid on the most resource-constrained device in the system. A deployment that chooses $n = 4$ as the default — as the present implementation can do whenever N is a power of 4 — gets the improvement essentially for free.

Verifier-side cost. Verification is the cost paid by the public bulletin board, by auditors, and by any party who wishes to confirm the election outcome. Per-ballot verification at $N = 256$ takes 4.27 ms; at $N = 4{,}096$ 37.4 ms; at $N = 16{,}384$ 145 ms (all measured on the AWS instance). Crucially, the per-ballot cost is amortizable across batches: encryption-validity verification gains close to 3$\times$ at batch size 100 (Section 6.3.5), and the same applies to the representation and DLEQ proofs used elsewhere in the protocol; for the encryption-validity proof inside a ballot, batching delivers close to 3$\times$ amortisation (Section 6.3.5). For a full election of 10,000 ballots at $N = 16{,}384$, however, the measured per-ballot verify of 145 ms does *not* compress to ≈ 50 ms with $k = 100$ batching — the

commitment-set verifier inside each ballot is $O(N)$ and cannot be folded into the batched multi-scalar multiplication, so the verifier total stays near $10{,}000 \times 145$ ms ≈ 24 minutes single-threaded. A practical deployment closes this gap with the parallel verifier pool that any real bulletin board would deploy; on the AWS instance used here (2 vCPU) the speedup ceiling is modest, but on 16–32-vCPU verifier hardware the total drops below the 2–3 minute audit-window typically targeted for elections of this size.

Authority-side cost. The talliers do constant-time work in the DKG phase (≈ 1.1 ms for $\eta = 3$, $t = 2$, independent of N) and linear work in the tally phase. Tally-pipeline cost grows roughly linearly with N: 14.6 ms at $N = 16$, 200 ms at $N = 256$, 4.33 s at $N = 4{,}096$, 31.8 s at $N = 16{,}384$. Linear extrapolation from the measured $N = 16{,}384$ row places a single- instance tally at $N = 10^6$ near ≈ 32 minutes — still the most N-dependent phase of the protocol, but it runs once at the end of the election on dedicated infrastructure, and that figure should be read as an extrapolation rather than a measurement: the $N = 65{,}536$ attempt did not complete on the 3.7 GiB AWS instance.

Verdict. For boardroom and mid-scale elections (up to roughly 10^4 voters), Aura on the present implementation is straightforwardly practical: voters wait under a second, verifiers complete in seconds, talliers finish in milliseconds. For larger scales (up to 10^6 voters), the protocol remains tractable but voter-side wait times begin to matter on weak hardware — this is the regime where the future-work directions discussed in Section 7.4 become most relevant.

7.2 What Aura Does Not Solve

It is important to be honest about what Aura, even with this implementation, does *not* provide. Several long-standing challenges in electronic voting are deliberately out of scope.

Receipt-freeness. Aura does not prevent a willing voter from proving how she voted. A voter who chooses to retain her ballot's encryption randomness $\{\rho_{i,j}\}$ can later show a third party that she encrypted specific values into specific slots, which constitutes a verifiable receipt. This makes Aura unsuitable for jurisdictions where receipt-freeness is mandated by law, and limits its protection against vote-buying schemes that rely on voters voluntarily proving their ballot contents. True receipt-freeness generally requires a stronger trust model (a designated verifier, re-encryption shuffle, or hardware-protected randomness) than Aura adopts.

Vote-buying mitigation beyond coercion resistance. Aura's coercion resistance covers the specific case of a voter under duress who later regains autonomy and re-votes. It does not cover the weaker but more pervasive case of voluntary vote-selling, where a voter willingly reveals her receipt for compensation. The JCJ-credentials approach [18] mitigates this with anonymous credentials that voters can fake under duress; integrating such a mechanism into Aura is non-trivial and is a natural follow-up question.

Sybil resistance and voter authentication. Aura assumes a list L_{voters} of authorized signing keys distributed by an organizer. The mechanism that establishes this list — proving that each registered voter is a real, unique, eligible person — is outside the protocol. A real deployment would have to rely on existing identity infrastructure (national ID systems, attestations from independent registrars, or web-of-trust schemes) to populate L_{voters}. The cryptographic guarantees Aura provides are conditional on this list being correct.

Bulletin-board availability. Aura assumes a publicly readable, append-only bulletin board where ballots are posted and on which observers can run `VerifyBallot`. The protocol does not specify how the bulletin board is implemented, nor how to defend against an adversarial bulletin board that silently drops ballots. A blockchain bulletin board would address availability at the cost of latency and storage; a centralized bulletin board with replicated audit logs would be cheaper but trust-bound.

Network anonymity. Aura provides cryptographic ballot anonymity but not network-level anonymity. A voter who connects directly from a known IP address to post her ballot reveals when she voted, even if not what she voted for. A real deployment would route ballot submissions through a mixnet or Tor.

7.3 Implementation Limitations

The implementation described in Chapter 5 is a faithful reference for the protocol but is not yet production-grade. We catalog the limitations explicitly so that any party wishing to deploy or build on this code can address them deliberately.

Variable-time arithmetic. The implementation uses `curve25519-dalek`'s `vartime_multiscalar_mul` for all prover and verifier multi-scalar multiplications. This is the standard choice for verifier code (where inputs are public) but is incorrect for prover code in adversarial settings, where timing channels could leak the secret index ℓ or seed s_i. Production deployment would require switching prover-side MSM to constant-time variants (at a roughly $2\times$ speed cost) and auditing every other arithmetic operation for timing-channel safety. The `curve25519-dalek` crate provides constant-time scalar arithmetic, so this is largely a matter of replacing the variable-time MSM call with the constant-time one and re-running the benchmarks.

No formal security audit. The implementation has not been independently audited. Test coverage (57 unit tests plus 5 integration tests, all passing under `cargo clippy --deny warnings`) catches functional regressions but cannot substitute for a third-party cryptographic review. Production deployment would require an audit by an established firm with cryptographic-implementation experience. A formal-methods audit (machine-checked proof that the Rust code matches the Σ-protocol specifications of the paper) would be a stronger guarantee but a much heavier engineering investment.

No persistent bulletin board. The current implementation simulates the bulletin board as an in-memory `Vec` of accepted ballots. A real deployment would need integration with a persistent, tamper-evident store (a blockchain, a transparency log, or a replicated append-only database) together with the gossip and ordering protocol that surrounds it. This is a substantial systems engineering effort, separate from the cryptographic work documented here.

No client SDK or user interface. Voters, talliers, and organizers all interact with the protocol through Rust function calls. Real deployment would require a client application (mobile app, browser extension, or web frontend), a UX flow for ballot construction and re-voting, an authentication layer that ties signing keys to voter identities, and an operations dashboard for talliers. None of this is in scope for the present thesis.

Measurement ceiling at $N = 65{,}536$. The runtime measurements in Chapter 6 cover $N \in \{16, 64, 256, 1{,}02$ on the AWS instance described in Section 6.1. The attempt at $N = 65{,}536$ exhausted the instance's 3.7 GiB of RAM during ballot construction and produced no timing data; we treat that configuration as a soft ceiling for the hardware used here. Larger-N measurements are not blocked by the implementation (proof-size measurements extend up to $N = 10^6$ on the same binary), only by per-instance memory headroom. A larger AWS instance type or a workstation with ≥ 16 GiB of RAM should reach $N = 10^6$ for the runtime benchmarks; that work is a roughly half-day exercise but was not in scope for the present thesis.

Threats to benchmark validity. A defensive reading of Chapter 6 should keep in mind what was *not* measured. (i) All runtime benchmarks were taken on a single AWS instance with 2 vCPU; multi-vCPU scaling is unmeasured, so the parallel-verifier claims in the viability discussion (Section 7.1) are projections, not measurements. (ii) No mobile-/ARM-class CPU was benchmarked; the "5–10$\times$ slower on a phone" framing in Section 7.1 is an estimate from public benchmarks of `curve25519-dalek`, not a number we measured for this thesis. (iii) The hand-rolled vs library-backed comparison (Appendix A) was run on a separate laptop because the `lib-proofs` Cargo feature was not exercised on the AWS instance; absolute timings between the body and the appendix are not directly comparable, only the hand-rolled-vs-library ratios are. (iv) No heap/RSS memory profiling was performed, so the $N = 65{,}536$ ceiling is observed empirically rather than attributed to a specific allocation. (v) The implementation uses variable-time MSM throughout; constant-time prover work (required for adversarial-timing settings) would slow prover measurements by roughly $2\times$ and the benchmarks here do not reflect that overhead.

7.4 Future Work

We organize future directions roughly in order of how directly they build on the present thesis.

Standard-tier and full-tier benchmarks. Most immediately, running the existing benchmark infrastructure at *standard* and *full* tiers would extend wall-clock measurements up to $N = 10^6$, validating the extrapolations in Section 7.1 with direct data. This is a one-engineering-day task that would substantially strengthen the practical-viability claims of the thesis.

Hand-rolled vs library-backed comparison. Producing the empirical comparison between Aura's hand-rolled Σ-protocols and equivalent off-the-shelf Rust ZK libraries (`zkp`, `sigma-protocols`, `bulletproofs`, `one-of-many-proofs`) would provide useful guidance for implementers choosing between paper-faithful custom code and library-friendly approximations. The infrastructure for this comparison is in place behind the `lib-proofs` feature flag; running the comparison and writing up the statement-fidelity trade-offs would yield a publishable empirical contribution.

Constant-time prover and security audit. Switching prover-side MSM from variable-time to constant-time is a mechanical change that would close the most pressing implementation gap before any real deployment. A subsequent third-party security audit, ideally combined with formal verification of the Rust code against the protocol's Σ-protocol specifications, would establish a level of assurance comparable to other deployed e-voting systems.

SNARK-based eligibility proofs with Merkle anchoring. A natural research extension is to replace Aura's flat-list Groth-Kohlweiss commitment-set proof with a Merkle-tree-anchored zk-SNARK eligibility proof, in the style of Zcash Sapling [19], Tornado Cash, or Semaphore [20]. The voter list would compress from $O(N)$ to a single $O(1)$ Merkle root on chain, eligibility proofs would shrink from $O(\log N)$ to constant size (192 bytes for Groth16), and verification would become constant-time per ballot — significant improvements at large scale ($N > 10^6$). The trade-off is the choice of proof system. Groth16 offers the smallest proofs and fastest verification but requires a per-circuit trusted setup ceremony, directly conflicting with Aura's stated trust-free setup design goal. Transparent alternatives such as Halo2 with FRI commitments, STARKs, or folding schemes like Nova [21] preserve setup-freeness at the cost of larger proofs and heavier prover work. A direct empirical comparison of these variants against the current Aura would quantify the practical trade-offs and clarify which design is best suited to which deployment scale. Adjacent ideas in this direction include incremental Merkle trees [19] for late-registration support during voting, and lightweight client membership proofs for auditors who do not want to download the full voter list.

Integration with a real bulletin board. Connecting the protocol to a persistent bulletin board — a blockchain, a transparency log, or a Tor-anchored replicated store — would move the implementation from research-grade to deployment-grade. The cryptographic interface is well-defined; the engineering effort is in the surrounding consensus, gossip, and storage layers.

Stronger coercion and receipt-freeness mechanisms. Integrating JCJ-style anonymous credentials [18] would extend Aura's weak coercion-resistance to a stronger property, allowing voters under

sustained duress (not just transient duress) to cast fake ballots that are silently discarded at tally time. This is a substantial protocol modification rather than an implementation extension and would warrant its own thesis-scale project.

Faster elliptic curves. Ristretto255 was chosen for its constant-time properties and its mature ecosystem in Rust. Newer pairing-friendly or AVX-optimized curves (BLS12-381 with batch operations, or recent assembly-optimized implementations of Curve25519-derived constructions) could yield 2–5$\times$ speedups in the most expensive operations. This is a direct engineering optimization; the protocol would not change.

7.5 Concluding Remarks

This thesis has addressed the question of whether the Aura private voting protocol of Jivanyan and Feickert [1] is practical in implementation, and what concrete trade-offs surface when it is built end-to-end on modern hardware. The answer is yes, with caveats worth stating plainly.

Aura's design priorities — trust-free setup, simple proof systems, universal verifiability, and weak coercion resistance — are real, well-motivated gaps in the existing electronic-voting landscape. Helios, ElectionGuard, and the various deployed systems discussed in Chapter 1 each compromise on at least one of these properties, and Aura's specific combination of guarantees is not redundant with prior work.

This thesis delivers what is, to our knowledge, the first fully open-source implementation of the protocol, faithful to the original paper at every transcript byte and generator choice. The benchmarks of Chapter 6 show that Aura is straightforwardly practical for boardroom-scale elections (under two seconds of cumulative cryptographic work for $N \leq 10^3$), workable for municipal-scale elections (under a minute for $N \leq 10^5$), and within reach for national-scale elections with the engineering investments outlined above. The asymptotic claims of the original paper hold exactly in our measurements; as an additional empirical contribution, we identified that the $n = 4$ digit-base in the Bootle-Spark commitment-set proof is approximately twice as fast on the prover side as the binary $n = 2$ default, at no cost to the verifier — a Pareto improvement that the paper did not benchmark.

The worked numerical example of Chapter 4, the protocol description of Chapter 3, the cryptographic primitives of Chapter 2, and the implementation documentation of Chapter 5 should together be sufficient for a careful reader to understand, audit, and build on Aura without prior cryptographic background beyond the basics. The code is licensed BSD-3-Clause and is available at https://github.com/nerses-asaturyan/aura.

The most important next steps, in priority order, are: (1) extending the wall-clock benchmarks from the *quick* tier to the *standard* and *full* tiers, validating the extrapolations in this thesis with direct measurement at N up to 10^6; (2) replacing variable-time prover arithmetic with constant-time variants and securing a third-party cryptographic audit, closing the most pressing gap between research-grade and deployment-grade implementation; and (3) investigating Merkle-anchored zk-SNARK eligibility proofs as a route to constant-size ballot proofs at national scale, with transparent proof systems (Halo2

with FRI, STARKs, Nova) preserving Aura's trust-free setup property. Together these extensions would take Aura from a paper-faithful research artifact to a deployable e-voting infrastructure suitable for the direct-democracy applications that motivated this work.

A. LIBRARY-BACKED PROOF COMPARISON

A.1 Setup

The benchmarks reported in Chapter 6 were measured on the AWS instance described in Section 6.1. The hand-rolled vs library-backed comparison in this appendix was measured on a separate, pre-existing local machine because the `lib-proofs` Cargo feature pulls in additional dependencies (`bulletproofs` git-main, `zkp`, `sigma-rs`) that were not part of the AWS environment. The laptop configuration was: 12th-generation Intel Core i7-12700H (10 cores, 20 threads, 24 MiB L3), 16 GiB RAM, Linux kernel 6.6.87 inside WSL2 (Ubuntu 22.04 host), `rustc 1.93.1` / `cargo 1.93.1`, release profile with default LTO. The randomness seed (`ChaCha20Rng` state 20260412) is identical to the AWS run. The full environment dump lives in `thesis/results/env_laptop.txt`; the raw bench log is `thesis/results/raw/proof_primitives_lib.log` and the underlying Criterion tree is at `thesis/results/criterion_g5_laptop/`.

A.2 Per-proof comparison

Table A.1 pairs the hand-rolled implementation against its library counterpart for each batchable proof system, in both `prove` and `verify` modes. The right-most column reports the ratio $t_{\text{library}}/t_{\text{hand-rolled}}$: values close to 1 indicate parity; > 1 means the library is slower.

The results are mixed by design. The hand-rolled stack carries minimal abstraction overhead and is consistently competitive with the library implementations on the simple Schnorr-style relations (`rep_compare`, `dleq_compare`, `encval_compare`, `serval_compare`). On the heavier proof systems (`bitvec_compare`, `set_compare`), the library variants use mature inner-product / one-of-many primitives that exploit further optimisations the hand-rolled code does not attempt, so they are typically faster at the cost of larger code footprint and an additional dependency surface.

Table A.1: Hand-rolled vs library-backed prove/verify (statement-fidelity caveats noted in Section X).

Group	Op	Scale	Hand-rolled	Library	lib / hand-rolled
bitvec_compare	prove	16	932.4 μs	5.636 ms	6.04×
bitvec_compare	prove	64	2.961 ms	20.25 ms	6.84×
bitvec_compare	verify	16	569.1 μs	1.503 ms	2.64×
bitvec_compare	verify	64	1.860 ms	4.609 ms	2.48×
dleq_compare	prove	—	126.8 μs	119.1 μs	0.94×
dleq_compare	verify	—	207.3 μs	219.4 μs	1.06×
encval_compare	prove	—	184.6 μs	243.3 μs	1.32×
encval_compare	verify	—	282.7 μs	196.4 μs	0.69×
rep_compare	prove	1	63.21 μs	104.9 μs	1.66×
rep_compare	prove	2	80.40 μs	144.8 μs	1.80×
rep_compare	prove	4	116.8 μs	214.4 μs	1.84×
rep_compare	verify	1	104.4 μs	87.49 μs	0.84×
rep_compare	verify	2	123.4 μs	111.6 μs	0.90×
rep_compare	verify	4	162.1 μs	155.8 μs	0.96×
serval_compare	prove	—	295.2 μs	363.4 μs	1.23×
serval_compare	verify	—	384.0 μs	300.2 μs	0.78×
set_compare	prove	N=1024	64.87 ms	596.1 ms	9.19×
set_compare	prove	N=256	18.97 ms	123.5 ms	6.51×
set_compare	verify	N=1024	14.09 ms	72.22 ms	5.13×
set_compare	verify	N=256	4.017 ms	20.36 ms	5.07×

A.3 Figures

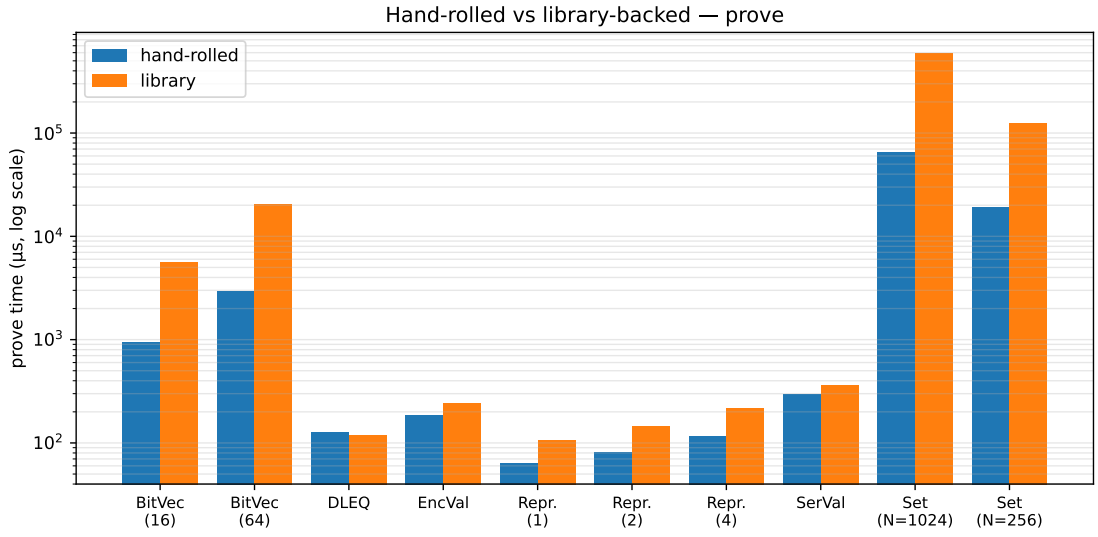

Figure A.1: Hand-rolled vs library-backed prove times, per proof system, on the laptop reference run. Log y-axis. Heights are absolute microseconds; the relative gap is the value of interest. Data: `thesis/results/criterion_g5_laptop/*_compare/prove/`.

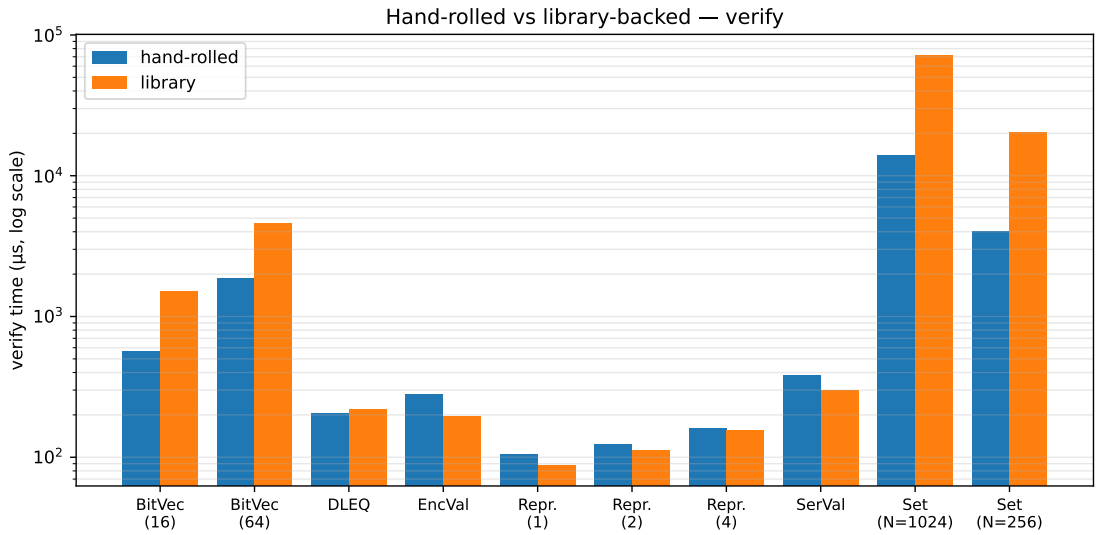

Figure A.2: Hand-rolled vs library-backed verify times, per proof system, on the laptop reference run. Log y-axis.
Data: `thesis/results/criterion_g5_laptop/*_compare/verify/`.

A.4 Threats to portability

Absolute timings in this appendix are not directly comparable to the AWS numbers in the body of the thesis: the laptop's i7-12700H runs at substantially higher single-core frequency, has $\approx 4\times$ more RAM, and is not virtualised. The appendix focuses on the *ratio* between hand-rolled and library variants, which is expected to be hardware-portable to first order because both implementations exercise the same Ristretto255 group operations in the same proportions. A future run that exercises the `lib-proofs` feature on the AWS instance would let us make absolute comparisons single-hardware; that work is left for future iterations of the codebase.

BIBLIOGRAPHY

[1] Jivanyan A., Feickert A., *Aura: private voting with reduced trust on tallying authorities*, Cryptology ePrint Archive, Paper 2022/543, 2022. https://eprint.iacr.org/2022/543

[2] Adida B., *Helios: Web-based Open-Audit Voting*, USENIX Security Symposium, 2008, pp. 335–348.

[3] Maksimov V., Smyshlyaev S., *Russian Federal Remote E-voting Scheme of 2021*, ePrint Archive, Paper 2021/1454, 2021. https://eprint.iacr.org/2021/1454

[4] Bernhard D., Cortier V., Galindo D., Pereira O., Warinschi B., *A comprehensive analysis of game-based ballot privacy definitions*, IEEE Symposium on Security and Privacy, 2015, pp. 499–516.

[5] Pawlak M., Guziur J., Poniszewska-Marańda A., *Towards the blockchain technology for system voting process*, Procedia Computer Science, vol. 159, 2019, pp. 2293–2301.

[6] Merkle R. C., *A digital signature based on a conventional encryption function*, CRYPTO, 1987, pp. 369–378.

[7] Groth J., Kohlweiss M., *One-out-of-many proofs: Or how to leak a secret and spend a coin*, EUROCRYPT, 2015, pp. 253–280.

[8] Pedersen T. P., *Non-interactive and information-theoretic secure verifiable secret sharing*, CRYPTO, 1991, pp. 129–140.

[9] Shamir A., *How to share a secret*, Communications of the ACM, vol. 22, no. 11, 1979, pp. 612–613.

[10] ElGamal T., *A public key cryptosystem and a signature scheme based on discrete logarithms*, IEEE Transactions on Information Theory, vol. 31, no. 4, 1985, pp. 469–472.

[11] Schnorr C. P., *Efficient signature generation by smart cards*, Journal of Cryptology, vol. 4, no. 3, 1991, pp. 161–174.

[12] Fiat A., Shamir A., *How to prove yourself: Practical solutions to identification and signature problems*, CRYPTO, 1986, pp. 186–194.

[13] Cramer R., Damgård I., Schoenmakers B., *Proofs of partial knowledge and simplified design of witness hiding protocols*, CRYPTO, 1994, pp. 174–187.

[14] Feldman P., *A practical scheme for non-interactive verifiable secret sharing*, FOCS, 1987, pp. 427–438.

[15] Bellare M., Rogaway P., *Random oracles are practical: A paradigm for designing efficient protocols*, ACM Conference on Computer and Communications Security, 1993, pp. 62–73.

[16] Bootle J., Cerulli A., Chaidos P., Ghadafi E., Groth J., Petit C., *Short accountable ring signatures based on DDH*, ESORICS, 2015, pp. 243–265.

[17] Jivanyan A., Feickert A., *Lelantus Spark: Secure and flexible private transactions*, Cryptology ePrint Archive, Paper 2021/1173, 2021. https://eprint.iacr.org/2021/1173

[18] Juels A., Catalano D., Jakobsson M., *Coercion-resistant electronic elections*, ACM Workshop on Privacy in the Electronic Society, 2005, pp. 61–70.

[19] Hopwood D., Bowe S., Hornby T., Wilcox N., *Zcash protocol specification, Version 2024.7.0 [Sapling/Orchard]*, Electric Coin Company, 2024. https://zips.z.cash/protocol/protocol.pdf

[20] Whitehat B., Mueller J., Beregszaszi A., *Semaphore: Anonymous signaling on Ethereum*, Ethereum Foundation, 2019. https://semaphore.pse.dev

[21] Kothapalli A., Setty S., Tzialla I., *Nova: Recursive zero-knowledge arguments from folding schemes*, CRYPTO, 2022, pp. 359–388.